

轻量开发专场

CANN开发者Meetup

2026.08.15 中国·南京



主办单位: CANN开源社区 协办单位:  AtomGit 开放原子紫金专区

CANN

CANNBot昇腾AI开发智能体概述

分享嘉宾：王星
2026.8.15 中国·南京



编码范式的百年演进：从指令到意图，对效率的追逐从未停滞



编码的终极畅想之一：手动编写 -> “言出法随”
我们离这个梦想还有多远？在昇腾算子开发领域，挑战有哪些？

挑战一：算子开发的综合挑战



设计

Design Complexity

- **经验依赖：**Tiling并行策略高度依赖专家经验，覆盖场景复杂；
- **硬件适配：**需要结合底层硬件架构特性进行优化，发挥计算潜能；



编码

Error-Prone Coding

- **领域知识断层：**通用大模型缺少Ascend C、CANN、昇腾专用编程范式的高质量训练数据；
- **编码门槛高：**依赖大量底层指令、内存调度、核间同步等细节；



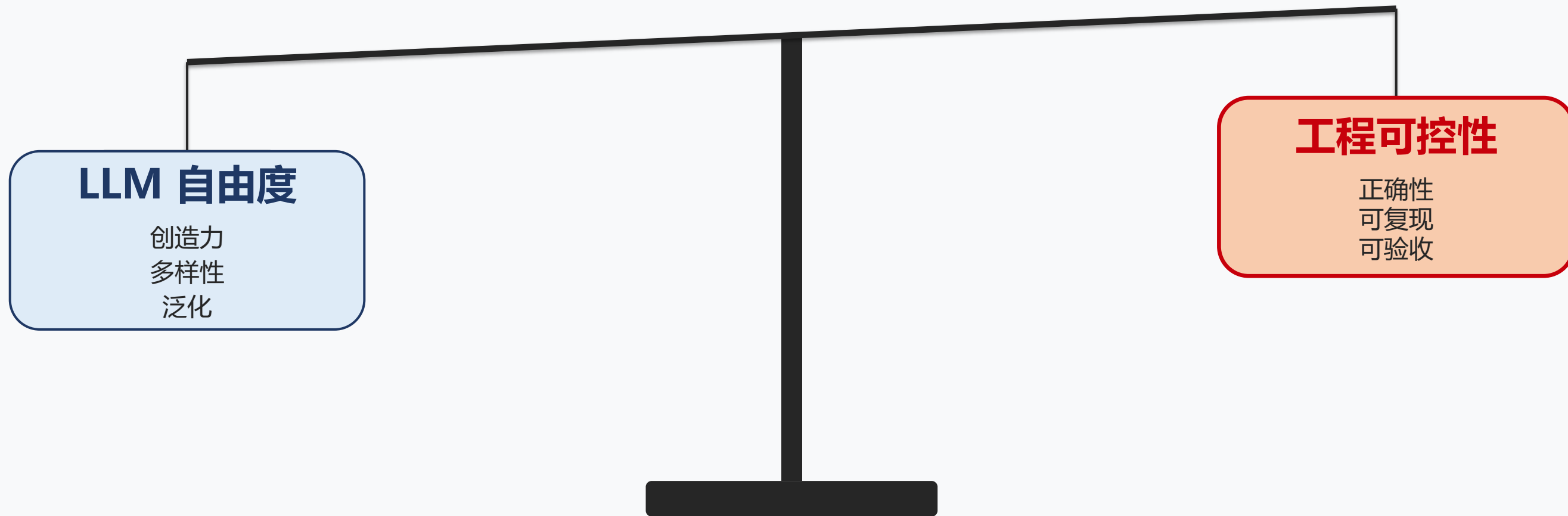
验证

Testing Complexity

- **功能与精度验证困难：**边界条件、异常输入、数值精度验证复杂；
- **性能调优难以闭环：**Profiling性能数据解析困难，难以自主完成“瓶颈定位→代码优化→再验证”闭环；

挑战二：算子AI生成的自由度 × 可控性得平衡

天平：左盘自由度 ↔ 右盘可控性



成熟的 Agent 系统：永远偏向可控（牺牲一点自由度换工程纪律）

CANNBot：把不可控的 AI 变成可治理工程系统的一套纪律

算子 (Operator)

算子是深度学习算法中执行特定数学运算的基础单元，例如加/减/乘/除等，通过组合这些算子，可以构建神经网络模型

CANNBot

CANNBot 是面向昇腾 CANN 开源社区构建的统一智能体平台，覆盖模型、算子、通信、图、编程语言、运行时 六大核心研发领域。包括代码生成，自主排障、闭环修复、端到端调优，将开发者从高门槛体力劳动中解放



Harness 工程体系 (The Skeleton)

| 自动化的骨架

CANNBot的自动化执行引擎和流程编排中枢，负责解析任务、分解步骤、调度Agent协同工作，确保复杂 workflow 顺畅运行



SKILLS 能力集 (The Soul)

| 智能的灵魂

CANNBot的“大脑”，是实现具体任务的AI Agent能力集合。包含“生成ReLU算子代码”、“分析性能瓶颈”等专业技能，是解决实际问题的核心



CANN-BENCH 评测基准 (The Ruler)

| 质量的标尺

面向昇腾平台的标准化算子与模型评测集，包含全面的性能、精度测试用例。它是保障产出质量、实现可信交付的基石

Harness: 长时运行 Agent 的有效驭具

- ◆ 想象一个软件项目由不同班次的工程师轮班开发，**每个新工程师上岗时都没有上一班的记忆**；
- ◆ 因为 context window 容量有限，且大部分复杂项目无法在单个 window 内完成，agent 需要一种方式来 桥接编码 session 之间的鸿沟；
- ◆ 解决方案的本质：把人类工程师交班的工程纪律编码进 Agent Harness；



任务解析与规划

充当“大脑”的角色，将用户的自然语言高层意图拆解为机器可理解、并可分步执行的原子化子任务序列。



技能调度与编排

充当“手脚”的神经末梢，动态调用 `cannbot-skills` 技能仓库中的相应模块，并根据任务逻辑合理编排执行顺序。



上下文管理

维护任务的全局状态，在不同技能模块间无缝传递信息，有效保持多步骤任务执行过程中的逻辑一致性与连续性。



异常处理与重试

提供高可用性保障，当执行流程中某一步骤发生失败时，能自动捕获异常，根据预设策略进行重试或回滚，提升任务成功率。

Harness核心命题：让 Agent “能力强大但行为可控”



大模型是天才，但不是员工



从“能对话”到“能干活”的三大鸿沟



可观测 × 可制衡 = 可控智能

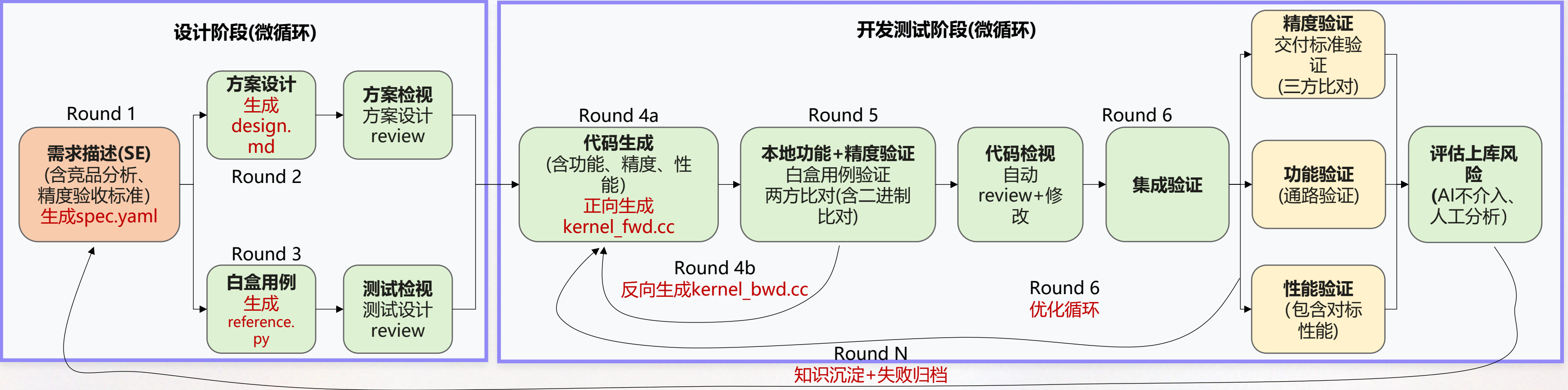
- **能力强大：**天赋极高但缺乏规则意识的天才实习生；不可控、易幻觉、无流程意识；
- **盲目乐观：**单个模型 ≈ 实习生天才，多个模型 ≈ 无人指挥的乐队；上下文焦虑，难以追溯调试

- **能编译但逻辑错误：**缺数学约束、缺数值稳定性技巧、缺对拍
- **逻辑正确但性能低下：**缺硬件约束、缺反模式审计、缺 profiling 闭环
- **性能达标但不可维护：**缺设计文档、缺 lineage、缺范式分类知识沉淀

- **越精确的约束**
- ↓
- **越小的搜索空间**
- ↓
- **越高的首次通过率**
- ↓
- **越低的修复成本**

Harness规划思路：从“天才实习生”到“模板正规军”

Round 0 工程初始化 创建 4 件套	Round 1 需求解析 生成需求描述 (spec.yaml)	Round 2 架构设计 生成方案设计 (design.md)	Round 3 测试设计 生成测试用例 (reference.py)	Round 4a 代码生成 生成 kernel_fwd.cc	Round 5 评估 + 门禁 编译+对拍+集成 测试	Round 4b 反向迭代 优化代码 (kernel_bwd.cc)	Round 6 性能优化循环 性能分析+自动优 化	Round N 沉淀归档 知识沉淀 + 失败 归档
-----------------------------	--	--	---	---	--------------------------------------	---	------------------------------------	------------------------------------



弹药链：9阶段流水线

拦截网：6道交付门禁

武器保险：7层约束体系

状态屏：跨Round状态管理

战场回传：5层反馈回路

作战手册：12条纪律

SKILLS能力集：沉淀资深工程师经验,构建标准化任务技能库



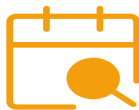
架构设计大师

自动设计最优算子架构和Tiling策略，从源头保障计算效率。



多语言编程专家

精通Ascend C, PyPTO 等主流算子开发语言，自动生成高质量、高可读性代码。



认真的代码审查员

基于行业标准自动进行多维度审查，严格把控代码的正确性与性能。



高效的调试工程师

利用自动化技术快速定位算子问题，并提供针对性修复建议。



专业的性能分析师

自动采集全链路性能数据，智能识别瓶颈点并提供优化方案。

开源目录

Plugins	Kernel 直调开发流程
ops-direct-invoke	算子注册调用开发流程
ops-registry-invoke	
Agents	
architect	方案设计
developer	代码开发
reviewer	代码检视
tester	代码测试
SKILLS	
npu-arch	NPU 架构知识与芯片映射
tiling-design	Tiling 设计方法论
api-best-practices	API 使用最佳实践
ops-precision-standard	算子精度标准
registry-invoke-to-direct-invoke	注册算子直调改造模板
direct-invoke-template	Kernel直调工程模板
precision-debug	精度调试与症状速查
runtime-debug	运行时错误码解析
ops-profiling	算子性能采集分析
st-design	ST 测试用例设计
ut-develop	UT 开发与覆盖率增强
code-review	代码检视规则
ascend-task-focus	长任务聚焦防迷失

SKILLS能力集：通过算子范式，构建算子领域“元素周期表”

27 范式 = 6 性质 × 平均 4- 5 范式 = 算子领域的元素周期表，覆盖主流 95%+，硬件映射确定性

基础 ×6

Elementwise · Broadcast · Reduction · Contraction · ArgReduce · LayoutTransform

控制类 ×3

MaskPredicate · SortSelect · Histogram

算法/形状 ×6

NumericalStable · Spectral · DynamicShape · VariableOutput · RandomSampling · Stateful

访存修饰 ×5

Padding · IndexGather · ScatterUpdate · AtomicUpdate · Interpolation

结构修饰 ×3

SlidingWindow · ControlFlow · Recurrence

数据/转换/通信/复合 ×4

Sparse · FusedComposite · Quantization · CollectiveCommunication



叠加规则(一个算子多标签)

softmax = Reduction + NumericalStable + Fused
conv2d = SlidingWindow + Contraction + Padding
flash_attn = Contraction + Reduction+NumStable +...



关键技术

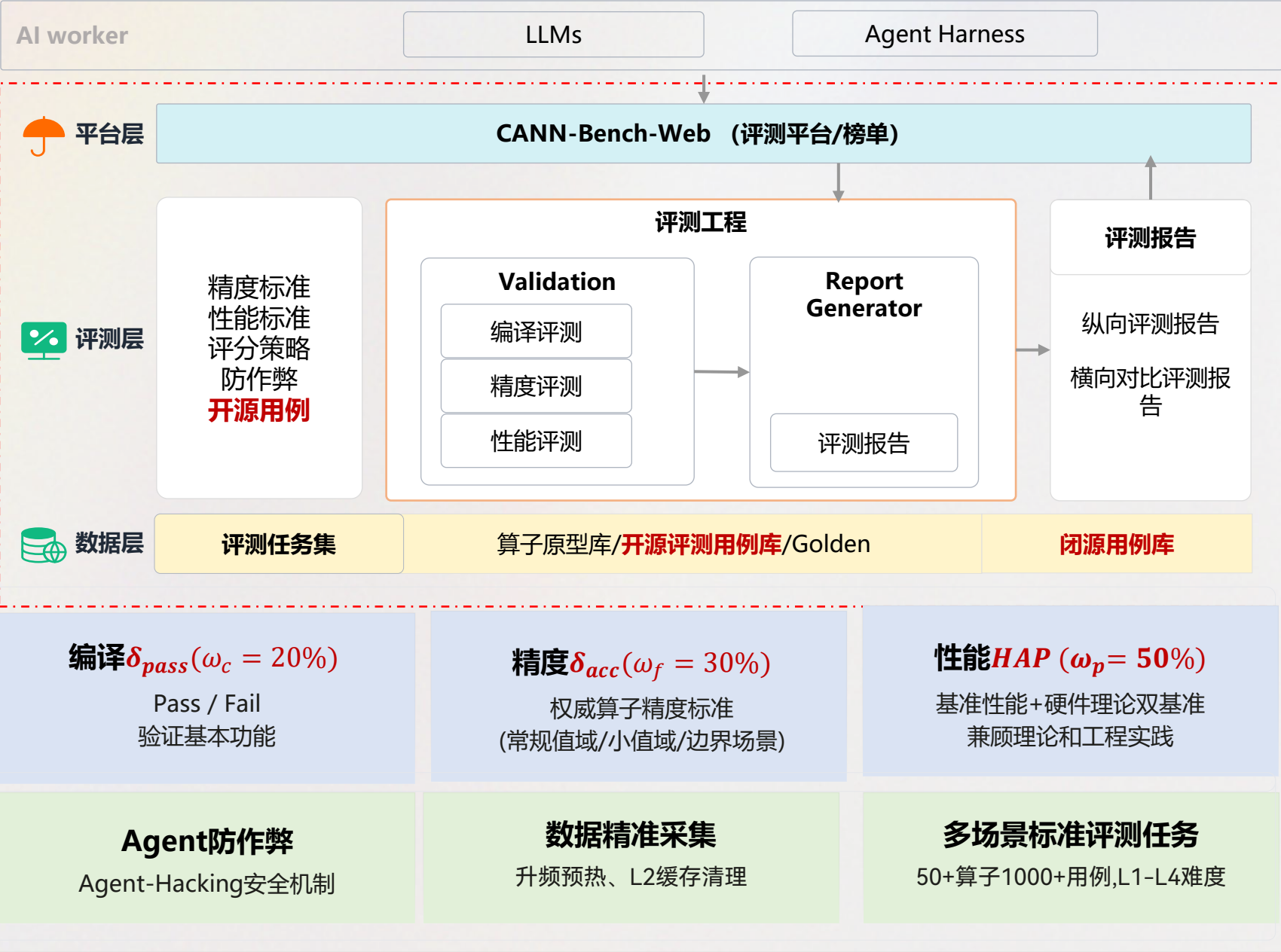
- 范式驱动设计，完成算子计算单元选型
- 专项skill覆盖全量范式
- 算子范式拼接，完成模板选型

CANN-Bench评测：构建算子生成标准化、工程化的评测体系

体系介绍

CANN-Bench是专为昇腾CANN平台打造的**自动化评测体系**。
通过**标准化**的流程、工具和平台，为AI模型和算子在CANN平台上的运行提供客观、稳定的评测基准。

科学的评分体系：
$$OperatorScore = \left[\omega_c \cdot \delta_{pass} + \sum_{i \in cases} \frac{\delta_{accuracy,i}(\omega_f + \omega_p \cdot HAP_i)}{|cases|} \right] \cdot 100$$



CANNBot：核心定位与设计理念总结

核心定位：昇腾原生AI开发智能助手

- 专为昇腾NPU平台深度定制，集成CANN生态核心能力
- 为开发者提供算子生成、模型迁移、部署调优的全流程功能

核心理念：Harness • Skill • Bench

- Harness：调度Agent协同工作，完成流程编排，提升Agent处理复杂任务的能力
- Skill： 统一标准化规划和管理，构建领域的“元素周期表”
- Bench： 构建Harness 流程与 SKILLS 统一上库标准，打造面向昇腾算子Agent生成质量的评测体系

THANKS



参与互动答疑贴赢取礼品



轻量开发专场

CANN开发者Meetup

2026.08.15 中国·南京



主办单位: CANN开源社区 协办单位:  AtomGit 开放原子紫金专区

CANN

CANNBot 算子智能生成

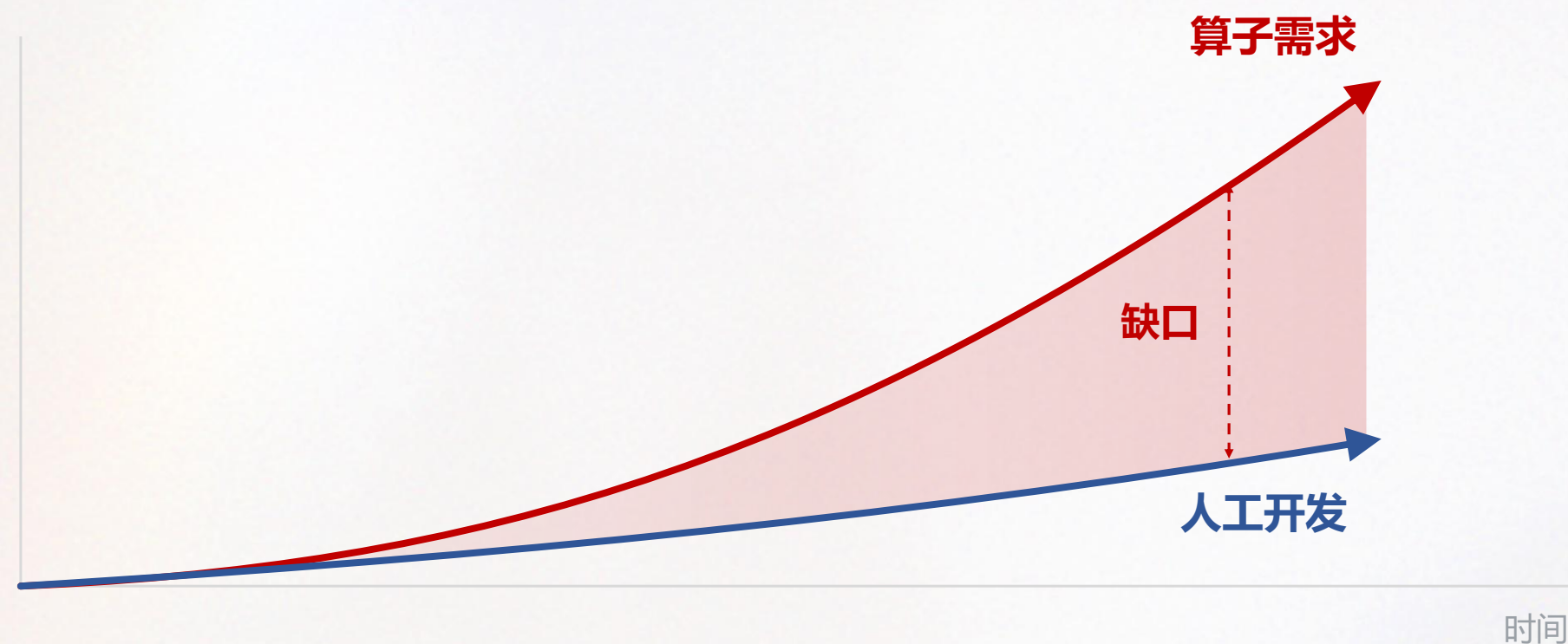
---基于 ops-direct-invoke-flash 的轻量算子开发

分享嘉宾：张子杰
2026.8.15 中国·南京



背景：新算子需求与人工开发成本

- 随着 DeepSeek、Qwen、Kimi 等新开源模型频繁发布，模型架构持续演进，各类 **算子融合 (Fusion)**、**精度适配 (FP8/INT4)** 和 **定制Kernel** 需求呈指数级增长。
- 人工开发一个高性能算子，从 算法理解 → 调度设计 → 调优验证，周期往往以 周/月 为单位。



CANNBot将Infra工程师从编码中解放，聚焦于更高价值的创新

使用方式：向CANNBot提出算子需求

首次安装

1 拉取代码

```
$ git clone https://gitcode.com/cann/cannbot-skills.git
```

2 进入插件目录

```
$ cd cannbot-skills/plugins-official/ops-direct-invoke-flash
```

3 初始化（以opencode为例）

```
$ ./init.sh project opencode
```

4 启动 CLI

```
$ opencode
```

5 触发 workflow

```
/ops-direct-invoke-flash
## 算子规格
- 算子名: swiglu
- 公式:  $y = x1 * \text{silu}(x2)$ , 其中  $\text{silu}(x) = x * \text{sigmoid}(x)$ 
- 输入:  $x1$ 、 $x2$ , 共 2 个, 逐元素 (elementwise) 运算, 同 shape
- 输出:  $y$ , 与输入同 shape、同 dtype
- 数据类型: float32
```

描述算子计算公式 & 规格

开发者描述意图，CANNBot负责落地

看缺口

用起来

看演示

拆内核

```
root@360940acb7de:/workspace# git clone https://gitcode.com/cann/cannbot-skills.git
```

拉取CANNBot代码仓，进入交互界面

看缺口

用起来

看演示

拆内核

算子生成 workflow Overview

阶段 0/1

准备

探测芯片型号
确认环境信息

阶段 2

编译骨架

初始化基础工程

阶段 3

规格设计

数学语义与边界值行为

阶段 4

实现设计

UB 预算 · 指令序列 ·
API选择

起步：环境 + 基础工程

设计：规格 + 实现

阶段 5

ST测试开发

测试驱动开发，完成测试用例集

阶段 6

Kernel实现

根据设计文档开发算子
Kernel

测试驱动开发

阶段 7

验收评审

重新编译 + 功能复测 +
合规扫描

阶段 8

文档

根据实现输出文档

验收与收尾

规格驱动设计，测试驱动开发，逐步验收

验证循环：明确的目标、达成路径和验收标准

目标

当前阶段的任务

准确的目标描述可以赋予Agent方向

达成路径

如何达成目标

达成路径指导Agent逐步推进

验收标准

判断目标是否达成

量化的验收标准可以让Agent低成本的判断目标是否达成

阶段 3 交付件

silu_gate_definition.md

definition-review • 7 项检查

阶段 4 交付件

silu_gate_design.md

design-review • 8 项检查

阶段 6 交付件

silu_gate.asc

编译前扫描 + 编译 + 真机 pytest

阶段 7 交付件

代码 + 文档 + 测试全部

acceptance-review • 独立重建复测

反馈回路

执行器

产物

重试

编排器

验收失败

验证器

AGENT编程：Human-in-the-Loop

自动化测试+手工验证

AGENT跑测试，人工确认结果

AGENT目标为生成测试报告而不是让测试通过，减少欺骗概率

高危操作+人工审批

AGENT申请权限，人工批准

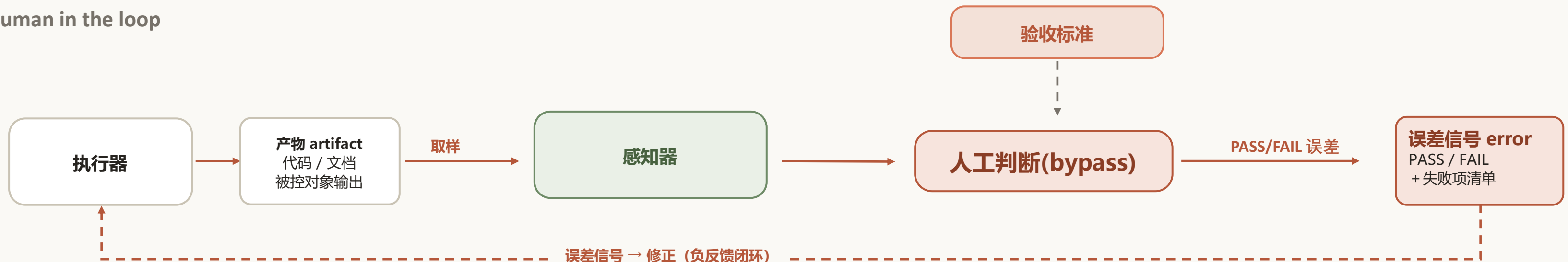
删除、重写、发布等不可逆操作，必须通过人工确认

歧义上报+人工决策

AGENT拿不准，人工决策方向

面对复杂的任务，提示词可能模糊、不准确、不自洽，需要人工及时纠偏。

Human in the loop



THANKS



参与互动答疑贴赢取礼品



轻量开发专场

CANN开发者Meetup

2026.08.15 中国·南京



主办单位: CANN开源社区 协办单位:  AtomGit 开放原子紫金专区

CANN

CANNBot模型迁移优化实践

分享嘉宾：陈宇
2026.8.15 中国·南京



- 01 体系概述与效果样例**
- 02 主流程编排与支撑约束机制**
- 03 使用方式与样例演示**

content

目录

体系概述与效果样例

01

模型Agent推理部署优化全景

核心思路：通过一个主流程把 NPU 推理优化，拆分编排成明确的步骤，按阶段推进、验证

🔗 开源仓库

- cann-recipes-infer 模型推理优化开源仓
<https://gitcode.com/cann/cann-recipes-infer>
- CANNBot Agent Skills 工具开源仓
<https://gitcode.com/cann/cannbot-skills>

📁 进阶优化 workflow

编排 model-infer-sota-approach

分析 model-infer-profiling
model-infer-perf-breakdown

优化 model-infer-multi-stream
model-infer-prefetch
model-infer-superkernel

🔗 当前基础部署优化 workflow 体系组成

1 编排层

model-infer-optimize
端到端总入口，统一编排分析、实施、验证闭环

2 核心层

model-infer-migrator
框架适配+部署基线

model-infer-parallel-analysis
并行策略分析

model-infer-parallel-impl
并行切分改造

model-infer-kvcache
KVCache+FA适配

model-infer-fusion
融合算子替换

model-infer-graph-mode
图模式适配

model-infer-quantization
量化接入+收益验证

3 调试层

model-infer-precision-debug
精度问题调试诊断

model-infer-runtime-debug
NPU运行时报错、卡死诊断

VALIDATION RESULTS

验证模型覆盖与效果样例

CANNBot模型Agent体系已在多种模型与场景中完成端到端优化测试

模型端到端优化测试

吞吐提升基于未优化的基线比较

Qwen3-0.6B 单卡, Dense GQA	8.9x Decode吞吐提升	Qwen3.5-0.8B 单卡, 混合架构	5.1x Decode吞吐提升
Deepseek-OCR 单卡, 多模态 MoE	3.2x Decode吞吐提升	Nemotron 单卡, Dense	3.3x Decode吞吐提升
DINOv3 单卡, ViT视觉	3.4x Decode吞吐提升	Gemma-4 8卡, MoE, 共享KV	5.4x Decode吞吐提升
LongCat-Flash-Lite 8卡, MoE+MLA	14.4x Decode吞吐提升	LongCat-Next 8卡, MoE+MLA	12.5x Decode吞吐提升

Decode吞吐提升8.9x：类似一条生产线，同样时间的产出提升到原来的 8.9 倍

单卡简单模型约2-3小时完成优化适配
多卡复杂模型约6-8小时完成优化适配

Hy3 优化结果样例

优化阶段	工作耗时	Decode单步	相对基线
模型分析+部署骨架	~0.3hr	-	-
并行切分方案与实施	~3.2hr	223.01ms	1.0x
KV Cache与FA改造	0.6hr	139.7ms	1.6x
融合算子接入	3.1hr	124.49ms	1.79x
图模式适配	1.7hr	37.83ms	5.89x
合计	~8.9hr	37.83ms	5.89x

说明：Hy3在A3 16die部署，序列长度4.5K，Batch Size 1

主流程编排与支撑约束机制

02

主流程编排

主流程编排将端到端优化组织为 6 个主要阶段，并最终输出优化报告

0 模型分析 + 基线建立

分析模型架构，完成部署并建立基线

1 并行化改造 (多卡)

分析并行策略，实施并行切分改造

2 KVCache + FA

完成KV Cache 选型与改造，适配FA融合算子

3 融合算子

分析匹配融合算子，实施替换并验证

4 量化适配 (可选)

接入量化产物并验证收益

5 图模式

图模式分析、断点消除与实施适配

模型部署优化 workflow



阶段详解：分工与角色派发

**Analyzer**
分析者

✓ 负责架构分析与方案设计

✓ 输出方案到 progress.md

✗ 禁止修改代码

**Implementer**
实施者

✓ 负责代码改造与修复

✓ 读写全部文件

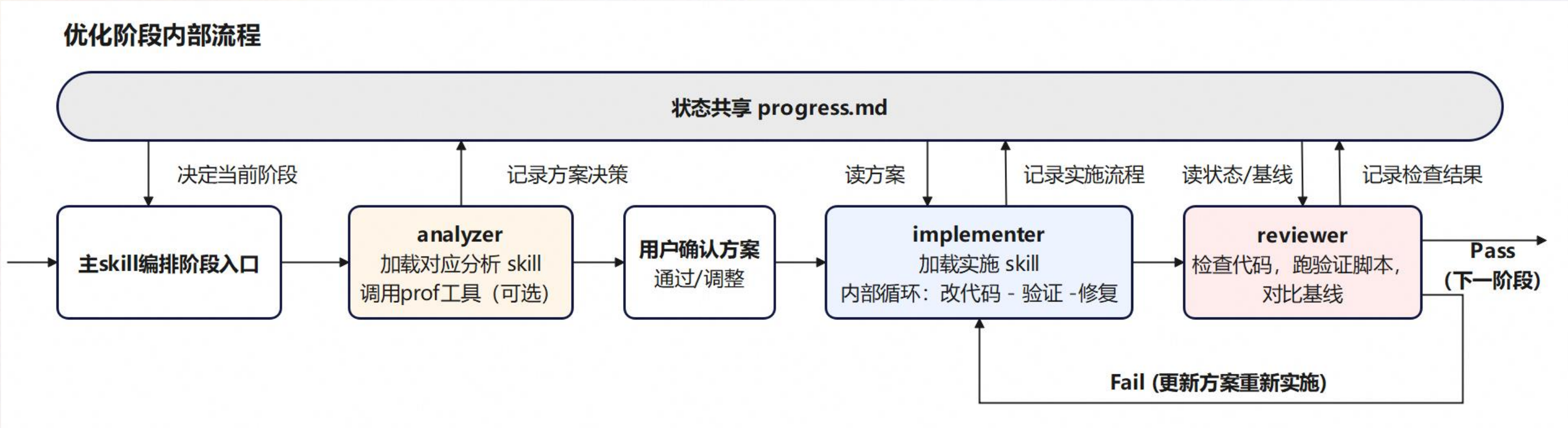
⚠ 必须完成自验证

**Reviewer**
验证者

✓ 负责代码审查与精度/性能验证

✓ 输出验证报告

✗ 禁改模型代码



Harness 约束体系



Harness主要作用：
约束 Agent 行为，使其在正确边界内执行任务

⚠ 实践中遇到的典型问题

验证报告显示通过，但修改代码从未被执行，Agent 编造技术解释而非调查

Agent 声称成功完成某项任务，但实际并未按要求完成



上下文约束

- ❓ **问题：**信息越多越容易忽略关键指令
- 💡 **方案：**AGENT.md，Skill主体精简，拆分流程至subagent执行



角色约束

- ❓ **问题：**Agent 跳过验证或越界改代码
- 💡 **方案：**按功能与权限拆角色，Hook 自动阻断越界



状态约束

- ❓ **问题：**上下文压缩丢失决策和调试记录
- 💡 **方案：**状态共享文档，归档+按需索引



流程约束

- ❓ **问题：**Agent 跳步执行，不遵循规定的流程
- 💡 **方案：**强制门禁+自验证检查

使用方式与样例演示

03

使用方式

1 方式一：infer仓库直接使用

通过 cann-recipes-infer 仓库直接使用，Skill / Agent / Hook 已内置在仓库

一键部署：bash scripts/init-agent.sh

支持：OpenCode、Claude 或 Codex

2 方式二：CANNBot 安装

通过 CANNBot 安装到任意已有项目，plugin 形式安装

安装方式：Plugin Marketplace / init.sh 脚本 / 手动安装

平台适配：Claude / OpenCode / TRAE / Cursor / Copilot / Codearts

两级别：project（项目级） / global（用户级）

▶ 启动模型Agent推理部署优化方式

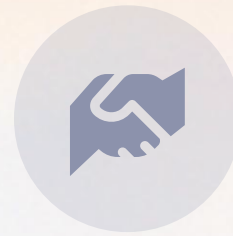
全流程优化（**推荐**）：“帮我完成 XX 模型NPU端到端适配及优化”

单阶段优化：“对 XX 模型单独做融合算子优化”

“对 XX 模型单独做图模式优化”

样例演示





欢迎社区参与共建



cann-recipes-infer
NPU模型推理优化开源仓



CANNBot (cannbot-skills)
Agent Skills 工具开源仓



CANNBot 与 Skill 开源交流群



试用



提 issue



补充案例与断点



参与社区讨论

THANKS



参与互动答疑贴赢取礼品



轻量开发专场

CANN开发者Meetup

2026.08.15 中国·南京



主办单位: CANN开源社区 协办单位:  AtomGit 开放原子紫金专区

CANN

CANNBot 智能代码审查

分享嘉宾： 谯竣仁

2026.8.15 中国·南京



01 为什么使用Agent进行代码审查

02 CANNBot代码审查功能介绍

03 CANNBot代码审查实操

content

目录

为什么使用Agent进行代码审查

01

为什么使用Agent进行代码审查

效率瓶颈与认知负荷

人工逐行阅读耗时极长，且随着审查行数增加，注意力迅速衰减，容易漏掉关键错误，成为开发流程的阻塞点

主观性与标准不一

审查结果高度依赖个人经验和偏好，容易导致“形式大于内容”（如纠结命名风格而非逻辑错误），且不同审查者对代码质量的判断难以统一

重复性劳动与一致性缺失

人工审查不擅长也不愿意处理大量重复性的、机械性的代码模式检查（如统一的空值处理、日志规范、重复代码片段）这类工作人工审查效率极低且容易因疲劳而漏掉

人工审核面临的困境



突破效率瓶颈与降低认知负荷

Agent 通过自动提取变更上下文并生成结构化摘要，在人工介入前完成初步缺陷拦截，大幅缩减开发者的阅读与理解时间

消除主观偏差与统一审查标准

Agent 基于预设的团队规范与架构原则进行客观评估，自动过滤格式争议，确保每次审查输出一致且聚焦核心业务逻辑

接管重复劳动与保障审查一致性

Agent 能够不知疲倦地执行跨文件的模式匹配与规范校验，并直接生成修复代码建议，彻底消除机械性漏检

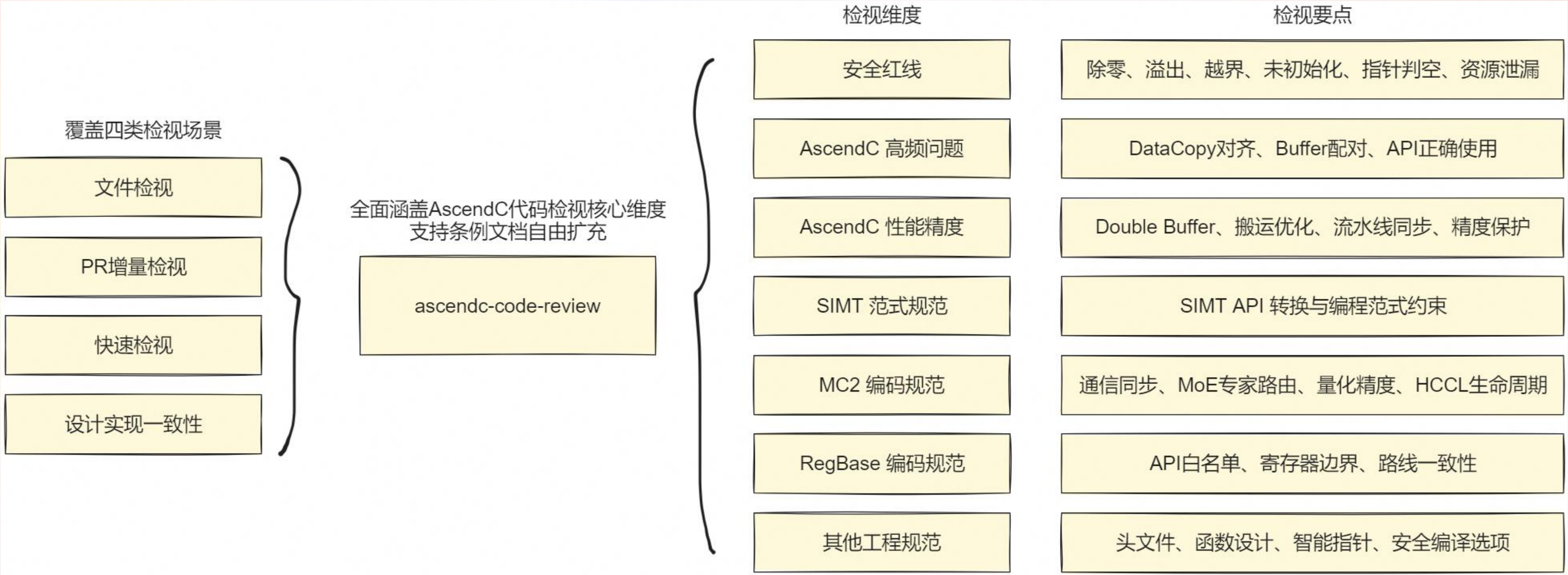
Agent的优势

CANNBot代码审查功能介绍

02

CANNBot代码审查简介

CANNBot 代码审查 Skill 是一个面向 Ascend C 算子仓（ops-transformer / ops-math / ops-nn / ops-cv）的智能化代码检视 workflow，基于"假设检验"方法论，将 200+ 编码规范条例转化为可执行、可追溯的自动化检视流程。



CANNBot代码审查——快速上手

CANNBot Skills 支持使用CLI工具进行可交互式一键安装，具体操作如下：

前置条件： 任选一种 AI 编程工具 —— OpenCode / Claude Code / Trae / Cursor / GitHub Copilot / CodeArts

快速安装（推荐，含 Skills + Agents + Workflows，适合端到端开发流程编排）

```
# Linux / macOS
```

```
curl -fsSL https://raw.gitcode.com/cann/cannbot-skills/raw/master/install.sh | bash
```

```
# Windows
```

```
iwr -useb https://raw.gitcode.com/cann/cannbot-skills/raw/master/install.ps1 | iex
```

然后运行 `install-helper` 启动交互式向导; 安装类型选择 Plugin -> 编程工具任选 -> 安装位置(推荐Project) -> 安装插件选: 代码检视

安装后使用： 启动 AI 编程工具，直接用自然语言描述需求即可触发，例如：

```
"帮我检视这个 PR: https://gitcode.com/cann/ops-transformer/pr/4046"
```

```
"快速检视 op_kernel/add_custom.cpp 有没有除零风险"
```

```
"对照 DESIGN.md 检查这份代码的实现一致性"
```

CANNBot代码审查——总体架构

渐进式披露
架构

Teams

CANNBot代码审查入口

CI服务支持

Subagents

结构性摘要

API 预研

文档DOC收集

条例派发ROUTER

专项检视

前置预研
并行检视

Skills

代码检视

文档搜索

性能优化

置信度机制

References

编码红线类问题

高性能规则

TopK类问题

Regbase领域规范

MC2领域规范

API使用规则

通用规范

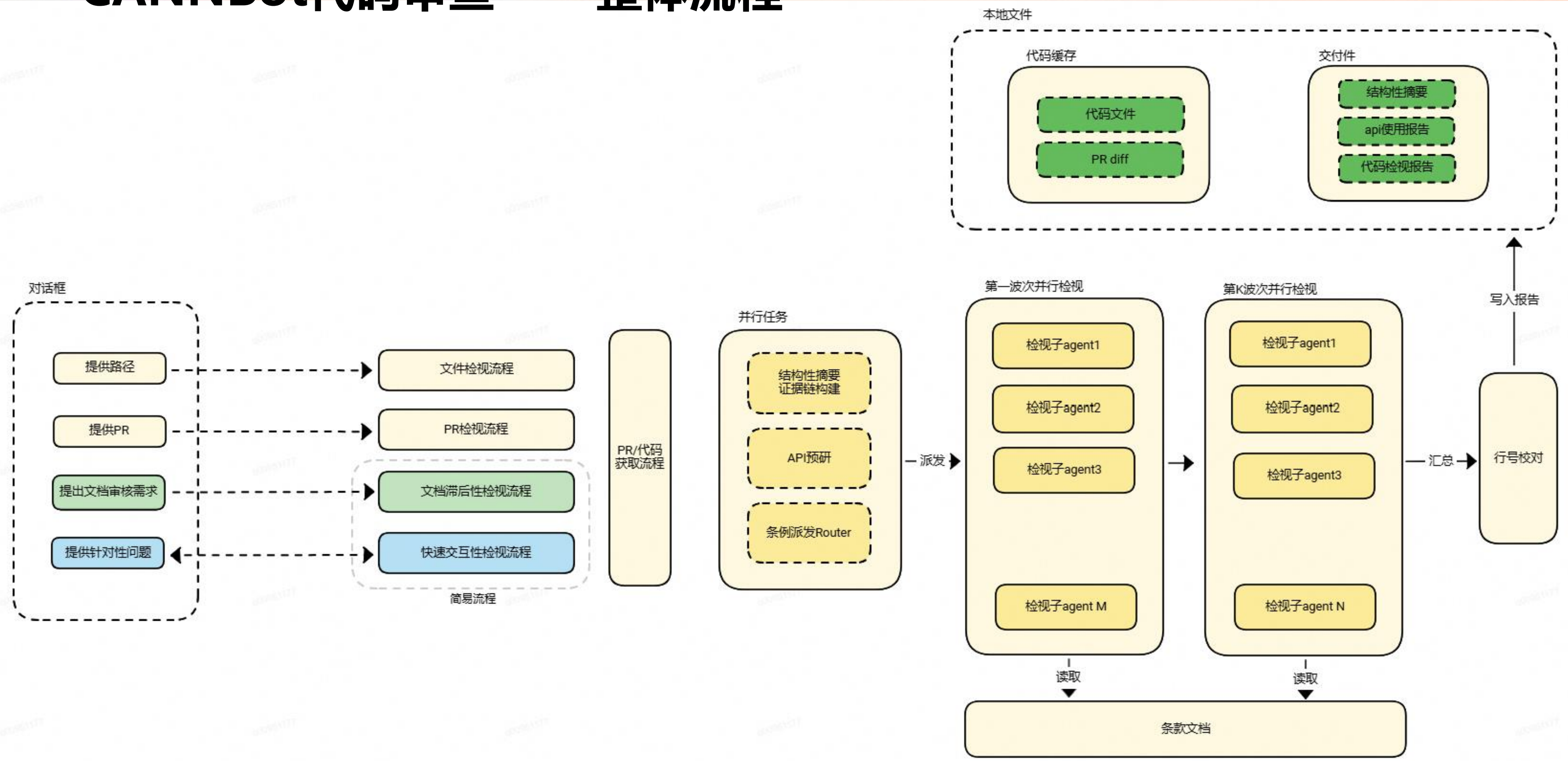
SIMT领域规范

CleanCode规范

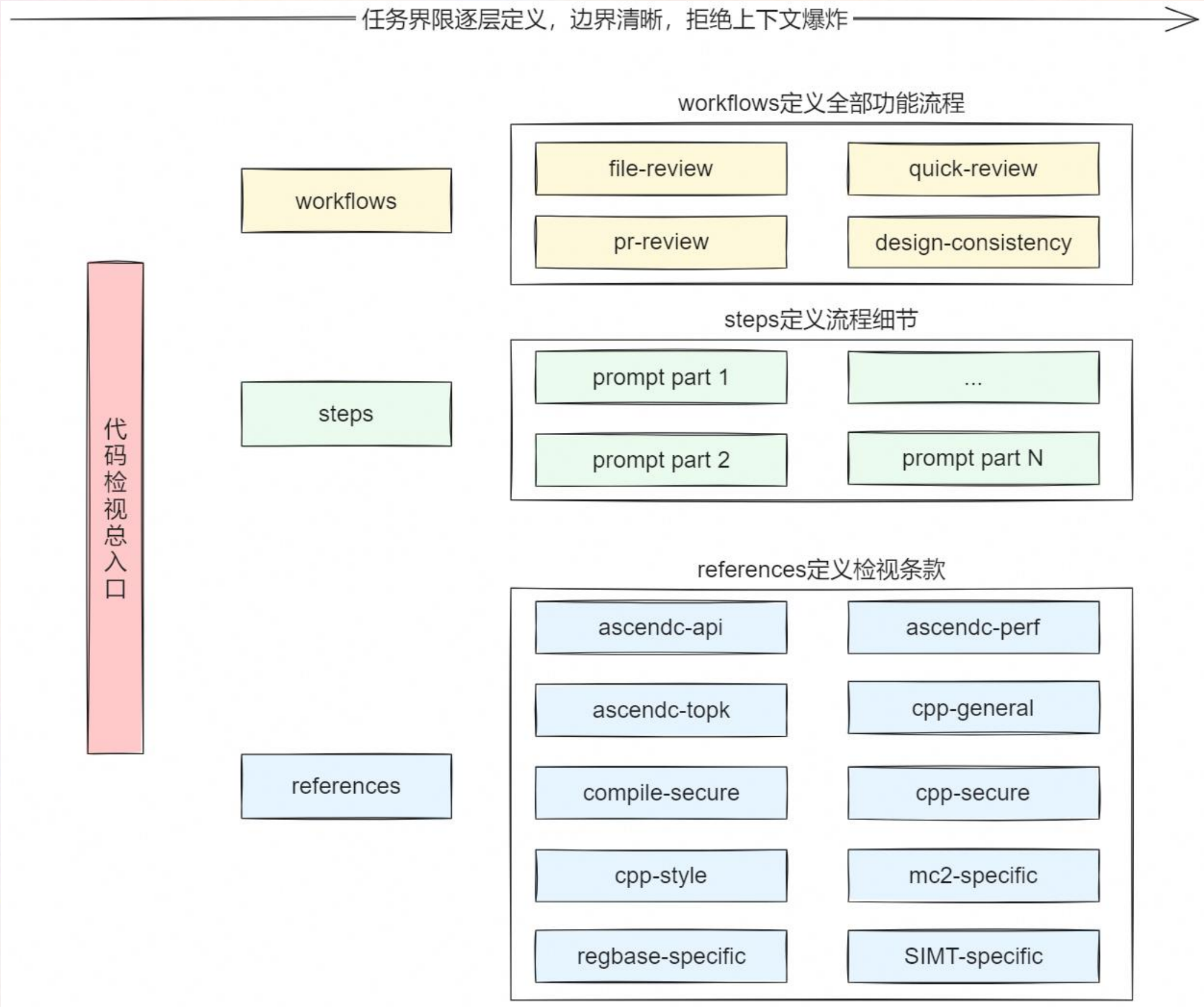
Python规范

规则自发现

CANNBot代码审查——整体流程

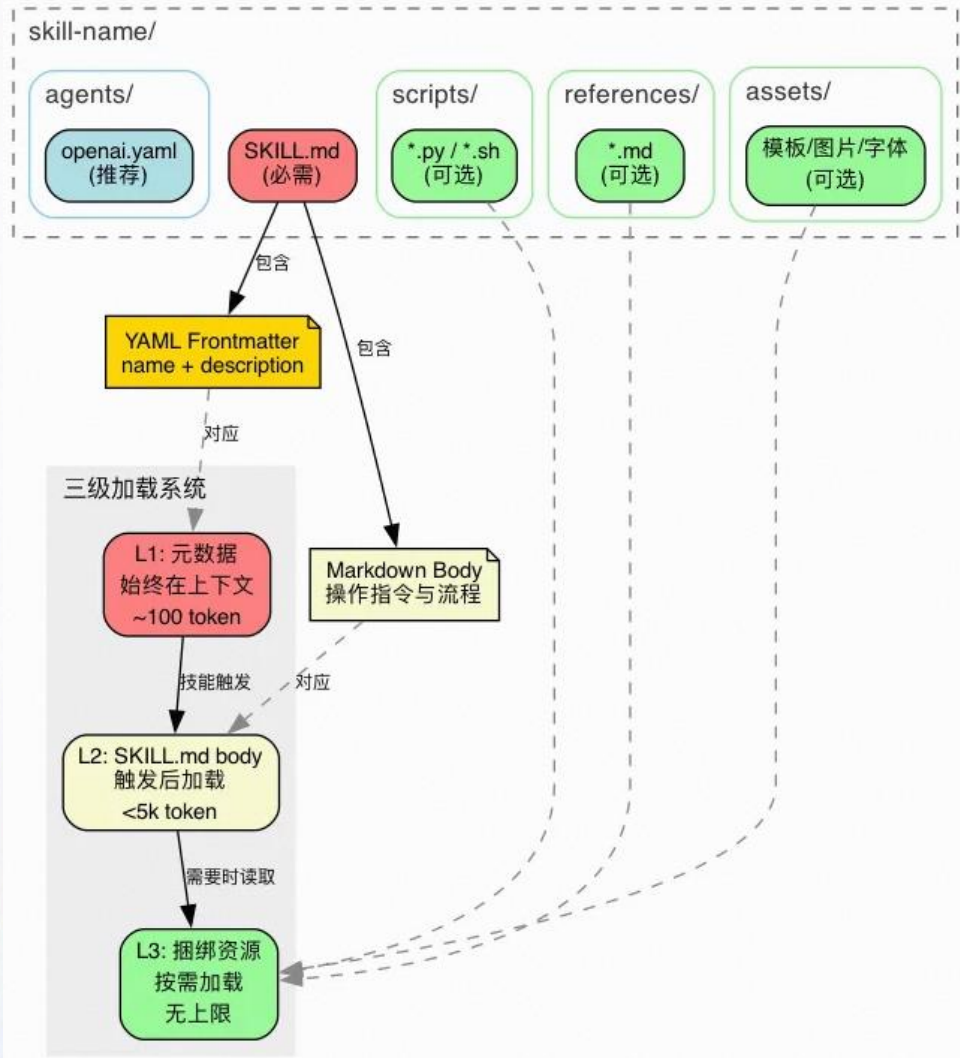


CANNBot代码审查——渐进式披露架构



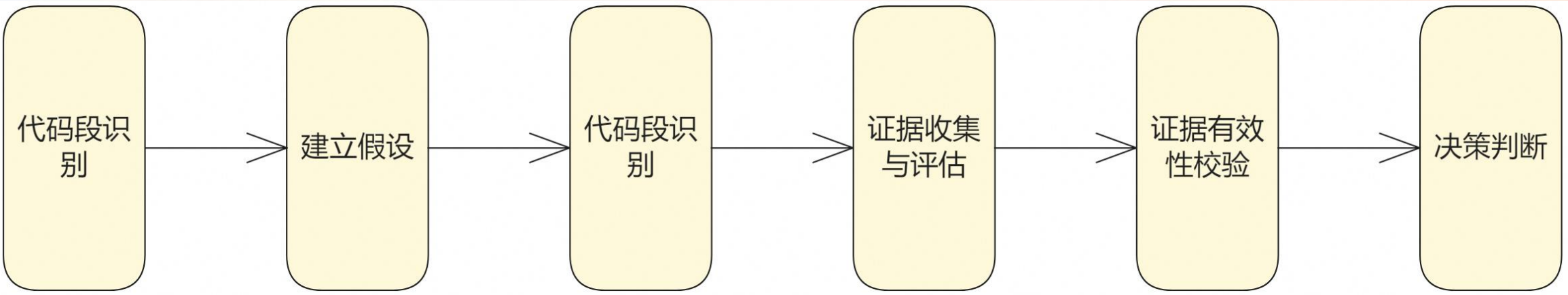
CANNBot代码检视Skill遵循流行的渐进式披露范式进行开发，**边界定义清晰，上下文分工明确**：

- 总体架构为DAG工作流
- 所有流程都由 workflows/ 定义
- 详细事件由 steps/ 定义
- 条款文档由 references/ 定义



CANNBot代码审查——置信度机制

CANNBot 代码审查遵循置信度分级制度，通过寻找证据链为代码做“置信度打分”，显著减少误报率



置信度机制

正向证据（增加置信度）：

证据类型	分值	分析动作
规范违反	+40%	对照规范条款识别违规点
上下文防御缺失	+30%	检查作用域内是否有防御代码
PR 归属	+20%	仅 PR 模式：问题代码在 diff 变更范围内
调用链风险	+15%	LSP/Grep 分析调用函数内部逻辑
数据流风险	+15%	变量来源不明确、运算路径不可控
领域关联	+10%	代码命中规则文件的领域触发特征

负向证据（降低置信度，反映已有安全保护。**单项分值较低，鼓励多条负向证据叠加才能显著降级**）：

证据类型	分值	分析动作
防御存在	-20%	当前作用域内有显式的零值检查 / 边界判断 / 非空校验等防御代码
上游校验	-15%	作用域外已有 OP_CHECK_IF / assert 等防护，且校验变量与风险变量相同
范围外	-50%	PR 模式：问题代码不在 diff 变更范围内

置信度分级标准

等级	区间	含义	分类
HIGH	80%+	明确违规证据	发现问题
MED	70-80%	可疑迹象需人工确认	需关注
LOW	<70%	证据不足，不判定违规	疑似

设计原则：负向证据单项分值低于正向证据，目的是防止 AI 偷懒——找到一条模糊防御就大幅扣分。真正充分防御的代码需要**多条负向证据叠加**（如同时满足「防御存在」+「上游校验」+「来源为编译期常量」）才能将置信度降至安全区间。单条负向证据（-15%/-20%）不应单独决定判定结果。

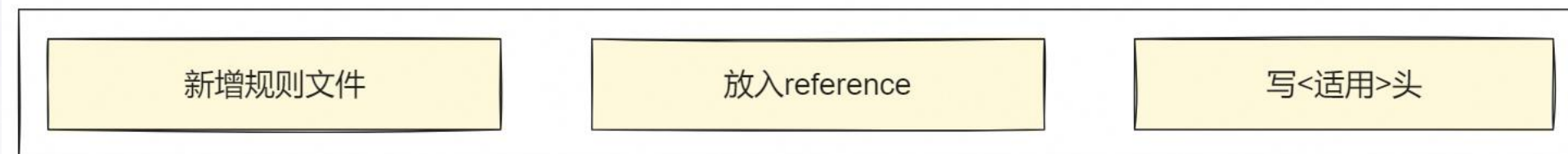
置信度分级

CANNBot代码审查——规则自发现机制

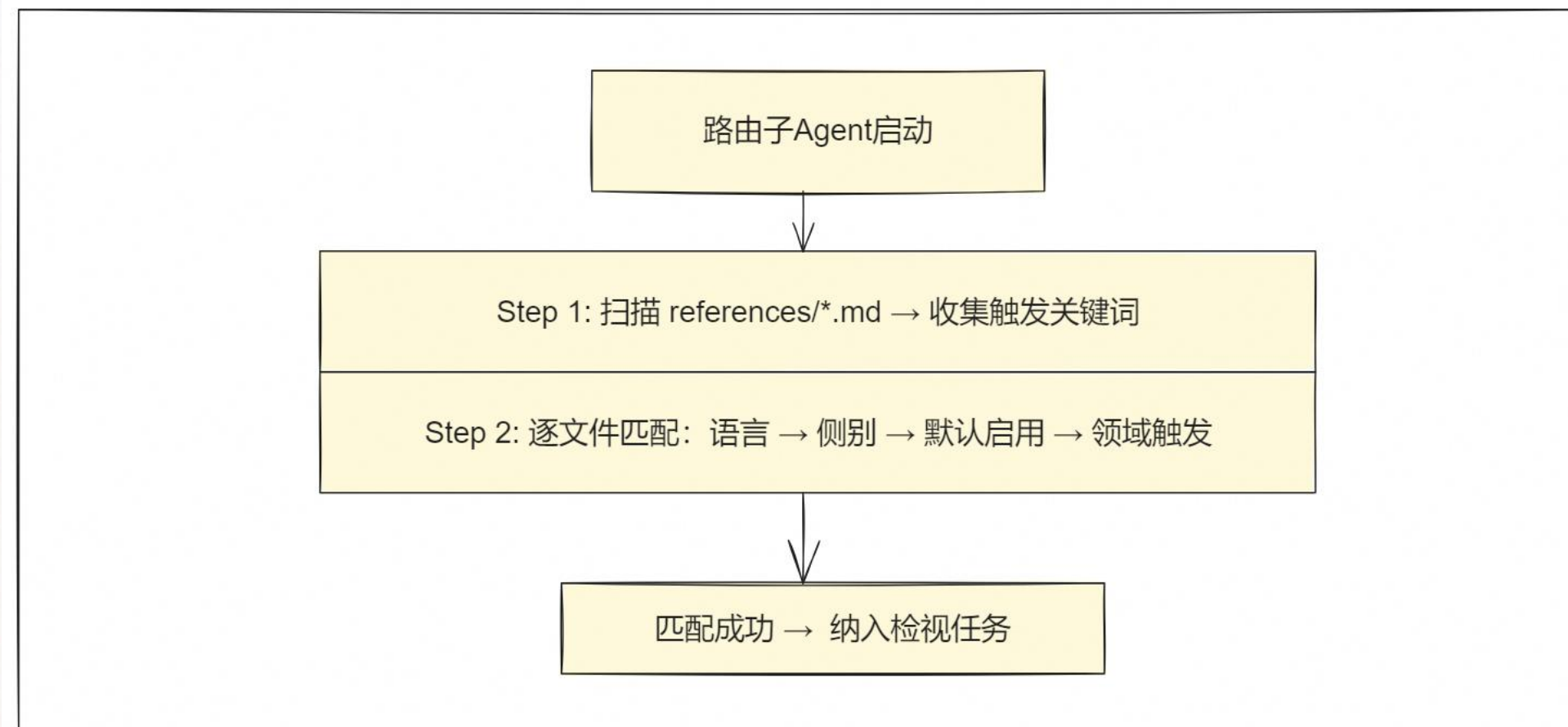
引入规则自发现机制，一方面拒绝Agent暴力检视所有规则条例，另一方面方便规则及条例文件的自由扩充

```
Preview Code 修改追溯 26.9 KB <> 源码
1 # Ascend C 红线 问题清单
2
3 <适用>
4 语言: C++
5 侧别: All, Host, Kernel
6 领域: true
7 默认启用: true
8 适用场景: Ascend C 算子开发中实际高频出现的编码红线问题，来源于生产环境经验总结
9 介绍: Ascend C 红线类 高频问题清单，10 条款覆盖算子开发中最常触发的错误模式
10 类别(All): 确保除法和余数运算不会导致除以零的错误、外部数据作为数组索引时必须确保在数组大小范围内、确保有符号整数运算不溢出、确保无符号整数运算不环绕、禁止使用未初始化的变量、指针操作使用前必须要判空、需要避免数据竞争
11 类别(Host): 无
12 类别(Kernel): 资源泄露防护、gm内存偏移或大小必须用int64表示、在支持superKernel场景禁止直接对gm地址进行读写(需要改用GetValue或SetValue)
13 > **说明**: TOPK 问题是检视实践中发现的高频风险点，需重点关注。条款标注适用范围: `[适用: All]` / `[适用: Host]` / `[适用: Kernel]`
14 </适用>
15
```

检视条款注册流程



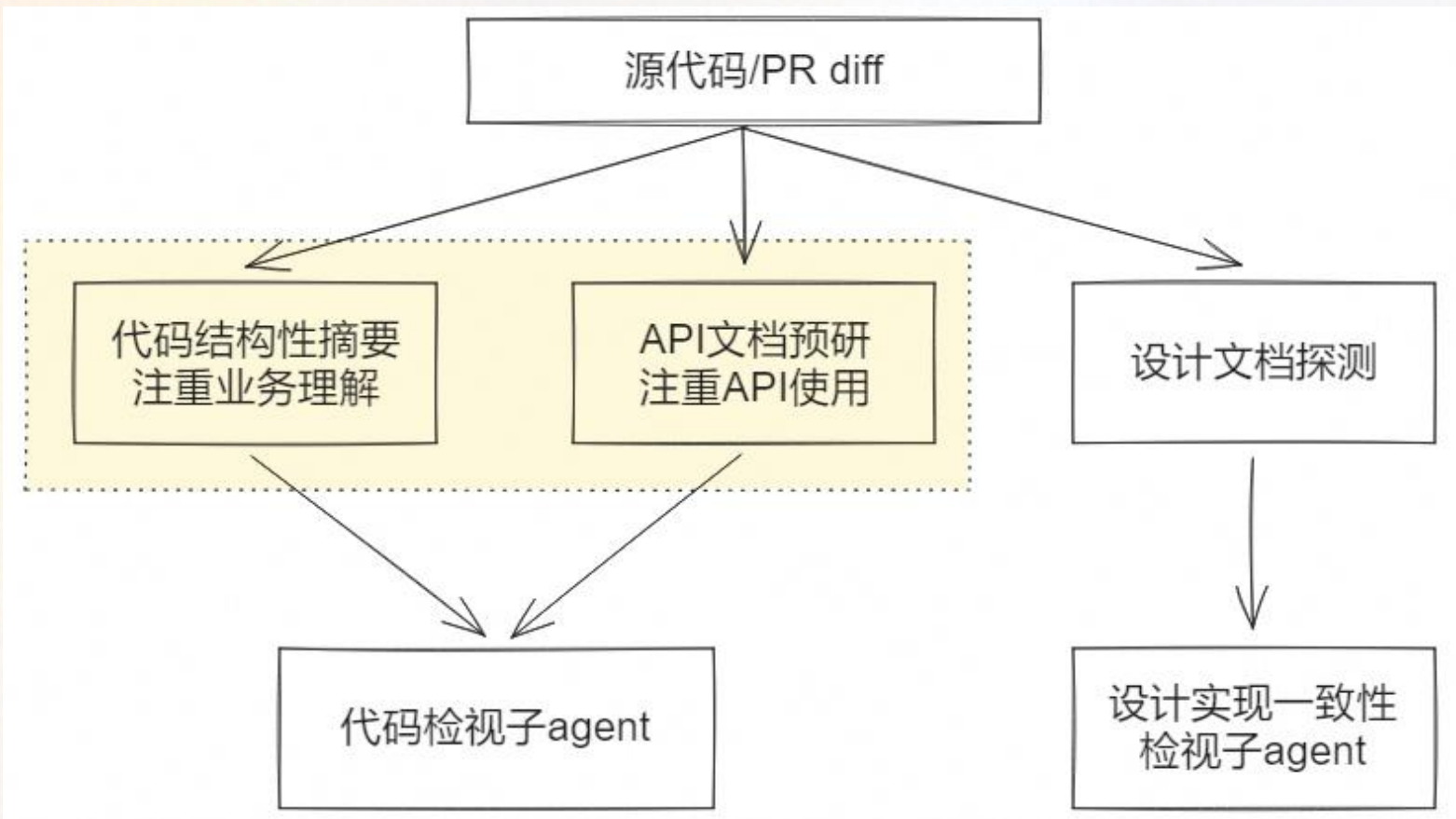
Agent 运行时流程



Agent自动根据条款文件<适用>Tag和代码特征自动算出“该检视哪些条例，怎么分组，分几波”等核心问题，从而实现条款自发现和检视内容的智能化

CANNBot代码审查——前置预研机制

预研行为的本质是**为检视子 Agent 降低注意力负担**——将通用但复杂的行为，以及需要宏观视角的行为，单独提取出来提前做。
检视子 Agent 拿到的是结构化、聚焦条例的现成知识+原始代码，**注意力得以集中在"按条例判断风险"这一核心动作上**



三路预研一览：

预研	模型	产出	减负效果
① 代码结构性摘要 code-summarize	general	代码脉络 / 侧别识别 / 变量溯源 / API调用索引 / 跨文件防御摘要 / TilingData值域 / 芯片架构参数 / 常量清单	假设检验证据直接查表，无需通读代码和 Grep
② API 文档预研 api-prestudy (仅 Kernel 侧)	opus	每个 API 的对齐要求、参数限制、配对规则、精度约束	API 类条例从每条例查 1 次文档降为 0 次，未覆盖时才补查
③ 设计文档探测 docs-detect	haiku	设计文档路径或空	找不到就不派 design-check，报告退化为纯条例检视

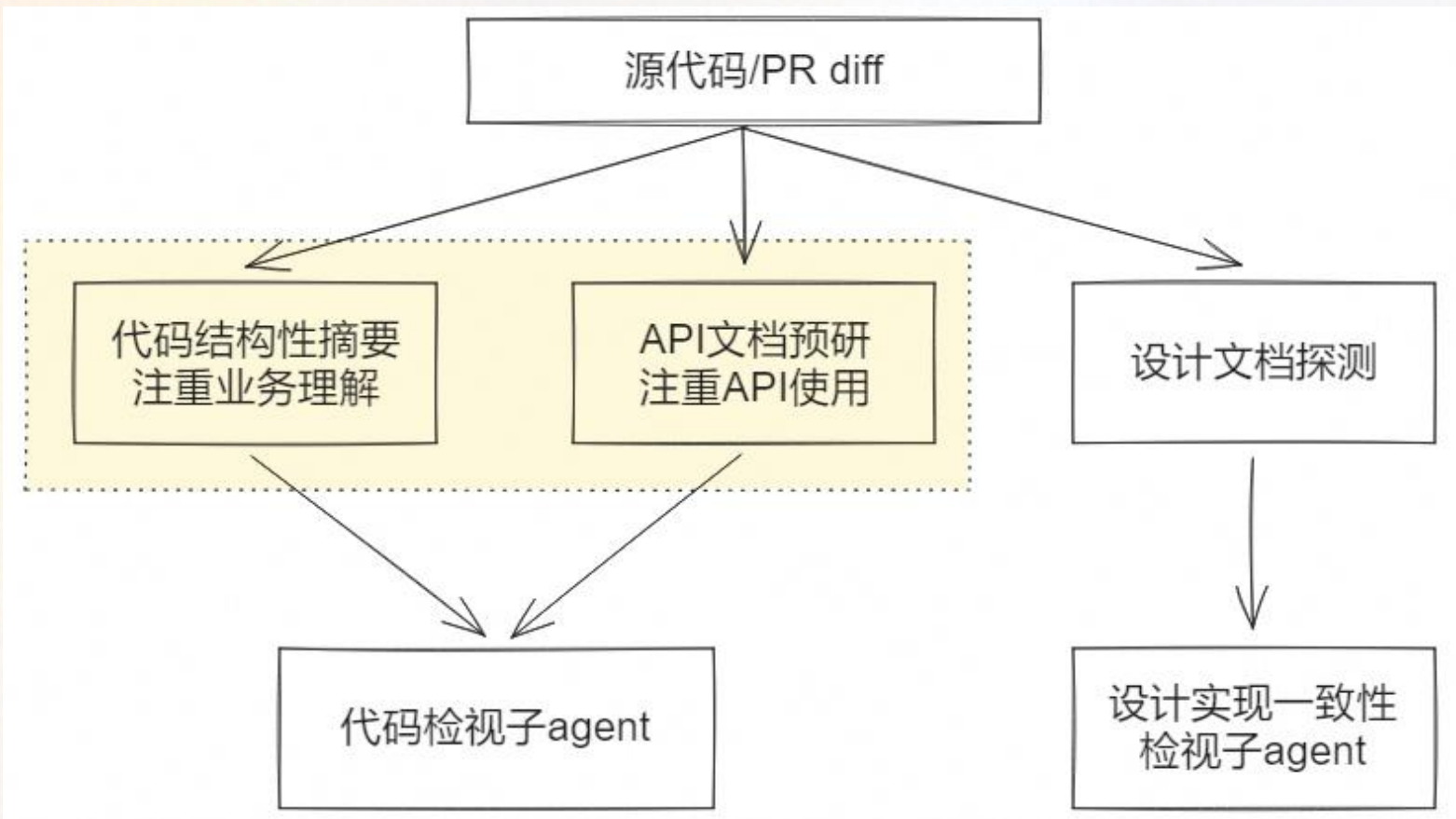
并行编排关键：

- 三路在**同一条消息**中并行派发，互不依赖
- api-prestudy 仅当侧别含 Kernel 时派发（纯 Tiling 侧跳过）
- 预研产出写入 `./operators/{name}/` 目录，检视子 Agent 通过路径 Read 复用

效果：检视子 Agent 启动时已有代码结构、API 约束、变量溯源三份现成知识，从“读代码+查文档+判断”三件事降为“只做判断”。

CANNBot代码审查——前置预研机制

预研行为的本质是**为检视子 Agent 降低注意力负担**——将通用但复杂的行为，以及需要宏观视角的行为，单独提取出来提前做。
检视子 Agent 拿到的是结构化、聚焦条例的现成知识+原始代码，**注意力得以集中在"按条例判断风险"这一核心动作上**



三路预研一览：

预研	模型	产出	减负效果
① 代码结构性摘要 code-summarize	general	代码脉络 / 侧别识别 / 变量溯源 / API调用索引 / 跨文件防御摘要 / TilingData值域 / 芯片架构参数 / 常量清单	假设检验证据直接查表，无需通读代码和 Grep
② API 文档预研 api-prestudy (仅 Kernel 侧)	opus	每个 API 的对齐要求、参数限制、配对规则、精度约束	API 类条例从每条例查 1 次文档降为 0 次，未覆盖时才补查
③ 设计文档探测 docs-detect	haiku	设计文档路径或空	找不到就不派 design-check，报告退化为纯条例检视

并行编排关键：

- 三路在**同一条消息**中并行派发，互不依赖
- api-prestudy 仅当侧别含 Kernel 时派发（纯 Tiling 侧跳过）
- 预研产出写入 `./operators/{name}/` 目录，检视子 Agent 通过路径 Read 复用

效果：检视子 Agent 启动时已有代码结构、API 约束、变量溯源三份现成知识，从“读代码+查文档+判断”三件事降为“只做判断”。

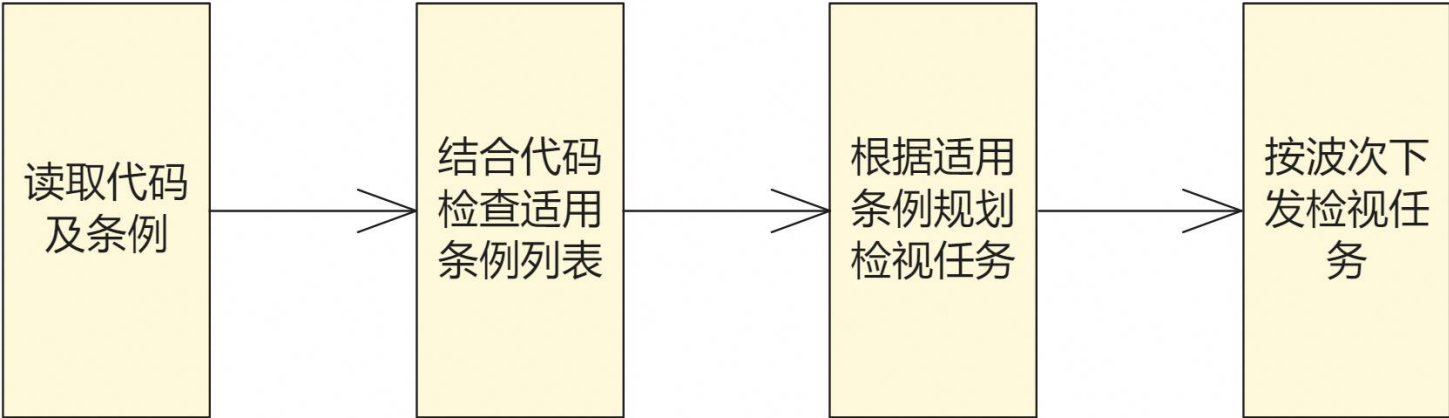
CANNBot代码审查——并行检视机制

📁 路由规划 [{极简/紧凑/标准}]
代码行数: {N}行 | 检视范围: {全量 / scope_hint}
代码语言: {C++/Python/Build/混合} | 代码侧别: {Kernel/Tiling/混合}
领域关键词命中: {命中列表}
匹配规则文件: {文件列表} (共 N 条例例)
跳过文件: {文件}: {跳过原因}
同类合并: {X 组跨文档合并}
内容过滤: 跳过 X 条不相关条例 (极简/紧凑模式)
文件分组: {G_file} 组 (多文件时列出各组文件清单, 单文件时省略此行)

波次1:
{组1 (来源, 侧别标签)}: 条例ID {标题}, ...
 代码范围: {仅Kernel / 仅Tiling / 全部}
 文件范围: {file_group_name} ({文件列表}) ← 多文件时填充, 单文件时省略此行
{组2}: ...

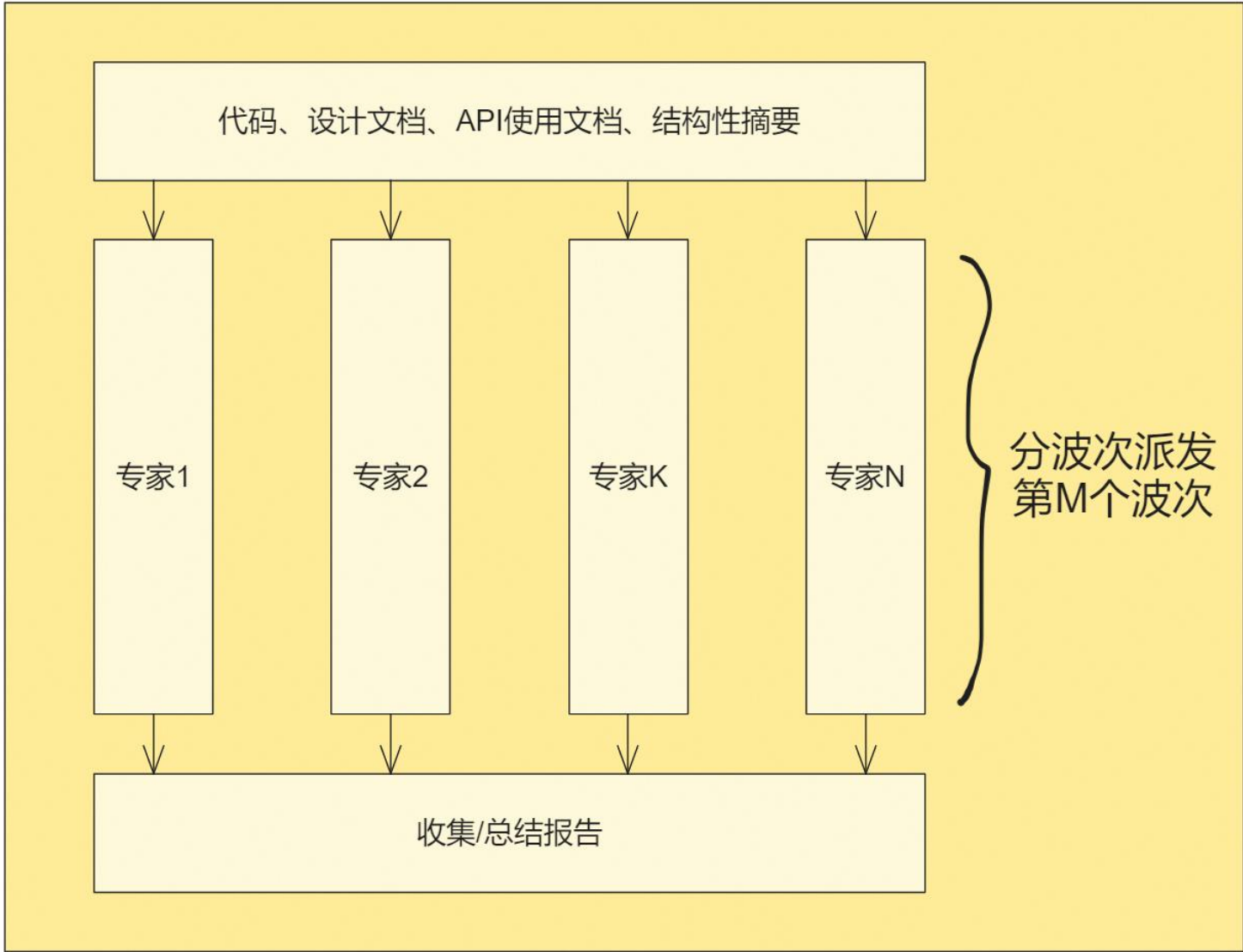
波次2 (如有):
...

共 G 组, 分 W 波。场景按每组的「代码范围」标签设置检视代码范围, 按「文件范围」确定检视的文件。



条例派发

200+ 条例不能塞给一个 Agent (上下文爆炸、注意力涣散、跳步偷懒)。按“小组策略 + 负载感知”分组, 每波并行派发 ≤10 个专家 Agent, 每个只看 2-3 条例 + 限定文件, 独立执行领域代码审查

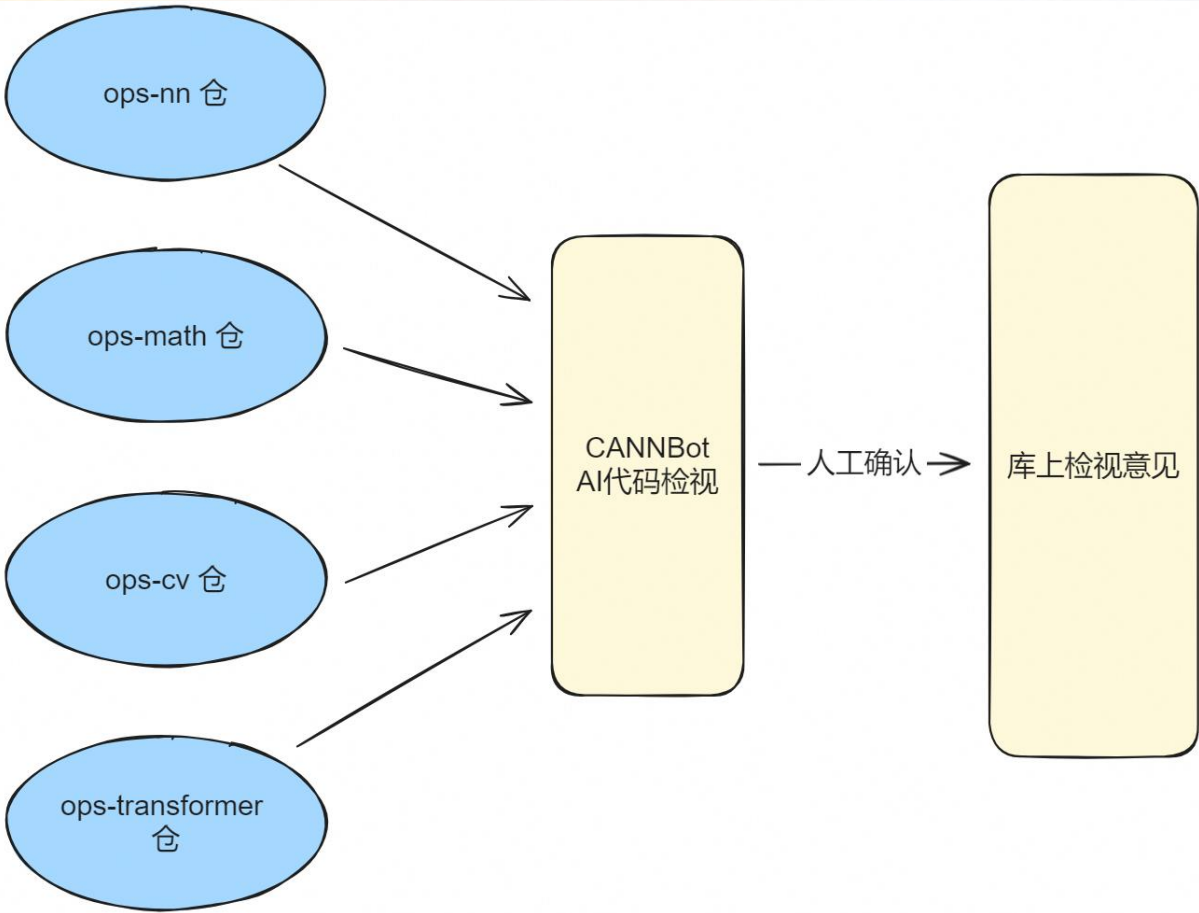


并行检视

CANNBot代码审查——CI轻松使用

CANNBot代码审查，支持在四大算子仓PR评论区使用指定命令轻松触发检视任务

- 使用 **cannbot-review** 命令触发检视任务
- 静待一段时间后，通过下载链接获取检视报告



GitHub repository: CANN/ops-transformer

Navigation: 代码 | Issues 149 | Pull Requests 305 | Actions | 项目讨论 | Wiki | 分析

Discussion: 8 | Submit: 1 | Check: 0 | File Changes: 2

Log:

- CANN CANN-robot 成员 1 天前 添加了label: **api-check-pass**
- CANN CANN-robot 成员 1 天前 删除了label: **ci-pipeline-running**
- CANN CANN-robot 成员 1 天前 添加了label: **ci-pipeline-passed**

anakinSkywalker 1 天前 评论:

cannbot-review

CANN CANN-robot 成员 1 天前 评论:

流水线任务触发成功
任务链接 [\[c6ab284fb37947778bd274592b23c599\]](#) [\[流水线指导\]](#)

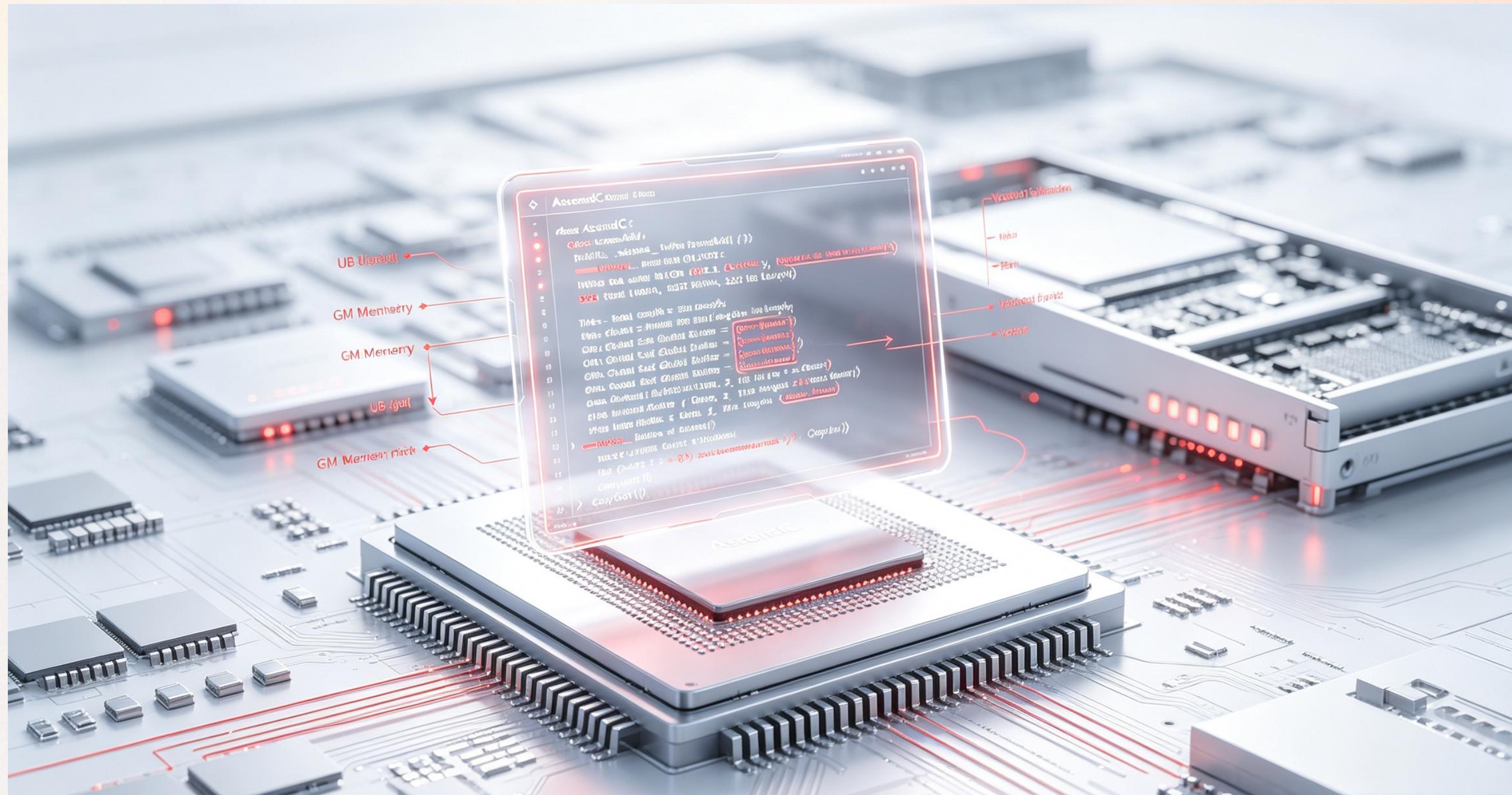
任务名称	状态	日志	下载链接
UT_Test_CANNBOT_Review	✓ SUCCESS		>>>>

[2026-08-04 17:23:41] CI执行结束

CANNBot代码审查实操

03

CANNBot 代码审核实操



THANKS



参与互动答疑贴赢取礼品



轻量开发专场

CANN开发者Meetup

2026.08.15 中国·南京



主办单位: CANN开源社区 协办单位:  AtomGit 开放原子紫金专区

CANN

CANNBot-Insight

---让Agent长链路工作流透明可审计

分享嘉宾：管兴华
2026.8.15 中国·南京



 **工具定位与核心愿景**

 **CANNBot-Insight 核心功能详解**

 **CANNBot-Insight的未来规划**

content

目录

工具定位与核心愿景

CANNBot-Insight 是专为 Code Agent 与大模型交互链路打造的可视化观测、诊断与评测工具。它把智能体交互的全流程拆开来看。能够将模型的思考过程、模型的输出、工具调用参数和结果，以及subagent调用，上下文使用等agent内部信息透明可视化。帮开发者链路透视，性能分析，问题定位，是保障智能体应用稳定运行、持续优化workflow的利器。

全链路交互可视化： 让智能体生产过程不再是“黑盒”

清晰呈现用户、Code Agent 和大模型三者的交互过程，还原对话上下文流、工具调用的细节，还有智能体的决策路径，让排查问题变得直观又高效。

模型问题诊断： 精准定位错误和性能瓶颈

精准溯源模型幻觉、上下文污染、指令遵循偏差这些核心问题，给出上下文治理的优化建议，帮快速找到根因，提升长程任务正确生成率。

工作流量化审计： 为流程优化提供数据支撑

对 Skill 编排、Workflow 自动化执行链路做全维度的量化审计，监控执行效率、节点成功率和资源消耗，生成多维度报告，为自动化流程迭代提供坚实的数据依据。

支持主流code agent工具

CANNBot-Insight 的启动流程非常简单，只需一条命令即可完成。同时，它支持多源数据导入，无需复杂配置，就能轻松接入 Claude、OpenCode 等平台的历史记录，开启智能分析工作。

```
root@LAPTOP-7F9TK24L:/home/guan/code/03_cannbot_insight/skills-dev/tests/cannbot-insight# bash start.sh
[setup] Advanced tabs: false (use -a flag to enable)
[start] Launching smart-agent (Python) on port 21026...
smart-agent server on http://localhost:21026
[start] smart-agent ready at http://localhost:21026 (PID 859)
[start] Launching CANNBot-Insight on port 21025...
[start] Waiting for server at http://localhost:21025...
▲ Next.js 16.2.9 (Turbopack)
- Local:      http://localhost:21025
- Network:    http://10.255.255.254:21025
- Environments: .env
✓ Ready in 814ms
```

Import Session

Import session data from local files or CANNBay

Source Type

opencode (SQLite) Claude Code (JSONL) CANNBot-Insight (SQLite) CANNBay

File Path

/

Browse

opencode (SQLite DB): ~/.local/share/opencode/sessions.db

Click Browse to explore directories and select files interactively

Load Sessions

Overview 全局总览面板

Overview

Turns

Trace

Context

Audit

Perf

Skills

Files

Tokens 421.1M

Cost \$0.00

Wall Clock 899.0min

Messages 3215

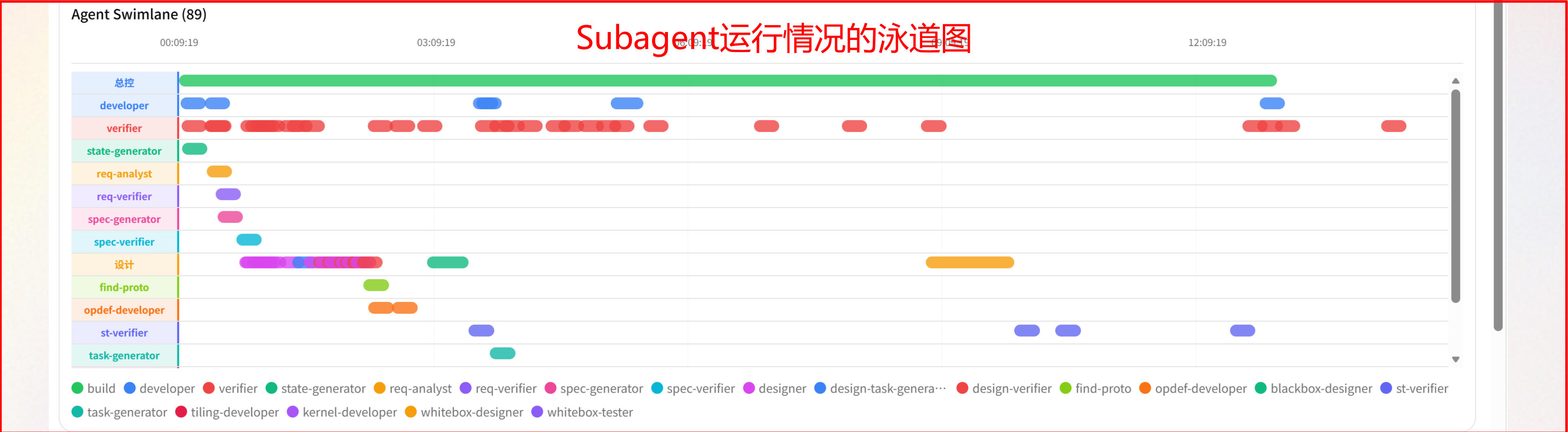
Tool Calls 3571

Skills 166

Subagents 88

Errors 116

任务的总览，包括token消耗，耗时，交互轮次，工具调用，skill加载，subagent等



Skills (16)

aclnn-st-framework1x89.4kt

ascendc-build-system2x260.1kt

ascendc-kernel-printf-dump1x178.3kt

ascendc-plog-analysis1x239.2kt

ascendc-regbase-best-practice15x248.3kt

ascendc-runtime-debug5x765.6kt

ascendc-st-design1x10.9kt

ascendc-whitebox-design1x15.7kt

ops-registry-invoke-glacier1x15.3kt

ops-requirement-analysis2x19.2kt

Skill加载情况

Turns交互轮次观测：深入对话细节的微观视角

OverviewTurnsTraceContextAuditPerfSkillsFiles

All (111) (1) (110)

Go to turnGo

选择turn

#0 user

"/ops-registry-invoke-glacier 帮我生成一个名为BenchHarnessCrossEntropyLoss的算子。算子信息位于can...
cannbot/glm-5.2

#1 assistant 2 tools ops-registry-invoke-glacier

<thinking>The user is asking me to generate an operator named `BenchHarnessCrossEntropyLoss`. The operator...
15.3k tokens 5.6s cannbot/glm-5.2

#2 assistant 1 tools

<thinking>Let me understand the skill. The `ops-registry-invoke-glacier` skill is an orchestrator skill. It says: > 你正...
21.6k tokens 54.3s cannbot/glm-5.2

#3 assistant 1 tools 1 subagent developer

<thinking>Now let me dispatch the developer to create the directory. The operator name is...
21.9k tokens 28.3s cannbot/glm-5.2

▶ developer Create operator dev directory

7 turns 82.8k tok 3.0s

#4 assistant 1 tools

<thinking>The developer created the directory. Now I need to dispatch the verifier to verify. Let me update the todo lis...
22.2k tokens 2.4s cannbot/glm-5.2

#5 assistant 1 tools 1 subagent verifier

22.4k tokens 15.4s cannbot/glm-5.2

LLM Input 1 messages, 15.3k tokens (1.5% context)

System (hidden) ≈15.2kt

user 96t

"/ops-registry-invoke-glacier 帮我生成一个名为BenchHarnessCrossEntropyLoss的算子。算子信息位于cann-bench-gen/tasks/level2/cross_entropy_loss目录下，包含以下文件（请仔细阅读）：
- proto.yaml：算子原型信息(重要)，包含算子分类、支持的数据类型、难度分类等
- desc.md：算子描述信息，包含数学公式、输入输出信息(重要)、精度标准等
- golden.py：算子对标竞品的标杆实现（torch/tensorflow）
允许参考：skills目录内的代码，当前工作目录下的参考代码。
禁止参考：当前工作目录外的、skills目录外的其他外部代码。

查看此轮时上下文的详细内容，
工具调用到技能加载

LLM Output <thinking>The user is asking me to generate an operator na... 185t reasoning + 108t output

thinking 185t ▶ show

text 108t

我将首先加载 `ops-registry-invoke-glacier` 技能，并并行读取算子信息文件。

Tool Calls (2, 1 skill)

ops-registry-invoke-glacier completed → Skills

bash completed ▶

Skills (1)

ops-registry-invoke-glacier invoke success 910t ▶

#1 assistant Context

root cannbot/glm-5.2

Context 1.5% → 2.2%

Minimal context — agent sending only recent tool output to LLM

User 1 msg 96t 0.0%

Cached 10.2kt 1.0%

System (hidden) 15.2kt 1.5%

Total: 1 messages 15.3k tokens

▶ Show full context

查看此轮时上下文的占有情况

Trace功能快速找到问题源头

Overview

Turns

Trace

Context

Audit

Perf

Skills

Files

设计文档

最近: 设计文档

搜索: "设计文档" 66 命中

source: 全部 用户引入 39 模型产出 10 子agent产出 17 thinking 23

tools: 全部 task 15 read 11 bash 6 glob 1

传播链 列表 传播图

起源 模型产出 Turn #81 root agent 工具参数 04:00:34

task: ...译通过; **仅定义结构与注册, 不实现 TilingFunc 切分计算** (DESIGN § 7 里程碑 1)。\n\n参考 设计文档: `docs/BenchHarnessCrossEntropyLoss/DESIGN.md` § 5.3\n\n完成后仅返...

View Turn → View Bridge →

模型推理

模型产出 Turn #83 root agent task 04:09:36

task: ...0)。期望值由**独立 oracle** (§ 5.2/ § 5.4 公式手算, 非被测 TilingFunc) 给出。 \n\n参考 设计文档: `docs/BenchHarnessCrossEntropyLoss/DESIGN.md` § 5.2/ § 5.4\n\n...

模型推理

模型产出 Turn #85 root agent task 04:29:35

task: ...) `恒 false`); DB=2、cacheBuf 16KB、dtype 不入 key 等工程约束逐条遵守。 \n\n参考 设计文档: `docs/BenchHarnessCrossEntropyLoss/DESIGN.md` § 5.2/ § 5.4\n\n...

模型推理

模型产出 Turn #90 root agent task 04:53:30

task: ...)/9 dtype。期望值由**独立 oracle** (§ 2 公式手算, 非被测 TilingFunc) 给出。 \n\n参考 设计文档: `docs/BenchHarnessCrossEntropyLoss/develop/design/DESIGN-BR...

模型推理

模型产出 Turn #92 root agent task 05:05:29

task: ...uction=0`、`isBatchReduce=false`、`isCMultiPass=false`)。 \n\n参考 设计文档: `docs/BenchHarnessCrossEntropyLoss/develop/design/DESIGN-BR...

模型推理

输入要追踪的关键字

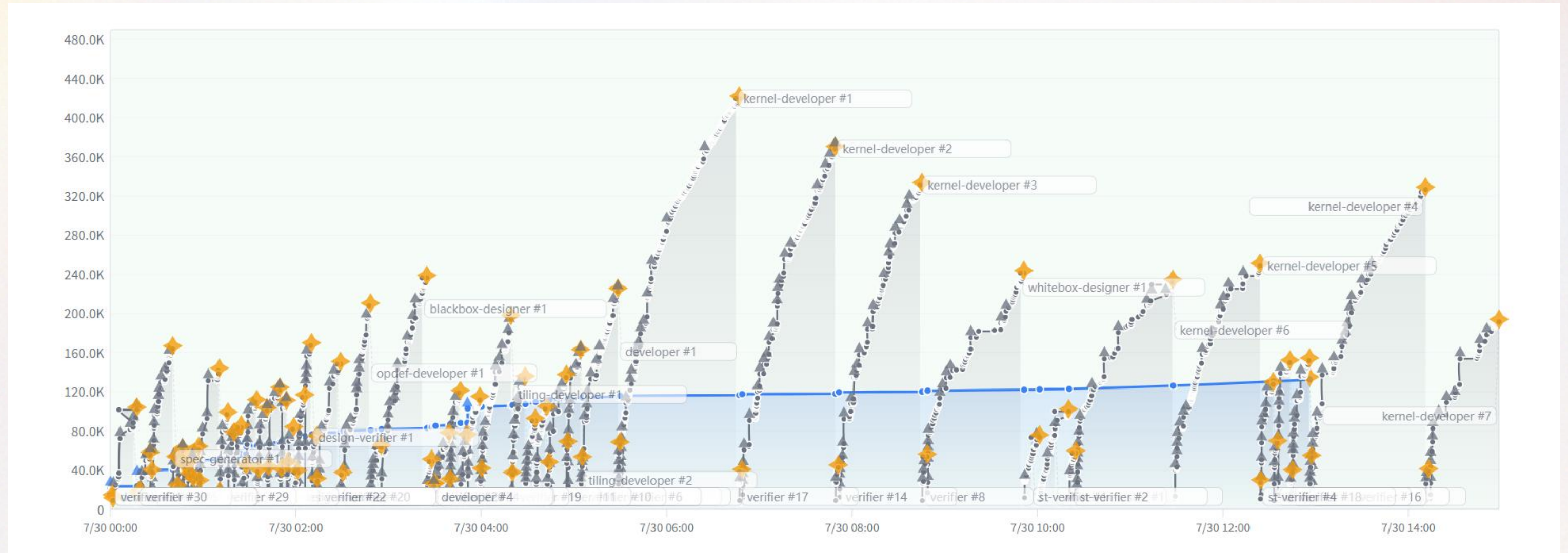
可以过滤来源, 工具类型

可以跳转具体轮次进行详细分析

在多轮对话、复杂Agent协作的场景里，Trace功能把原本看不见的信息流转过程彻底显性化。是排查幻觉问题的利器，更是优化Prompt工程、梳理Agent分工逻辑的关键工具。通过全链路追踪，开发者能快速定位问题根因，大幅降低调试成本。

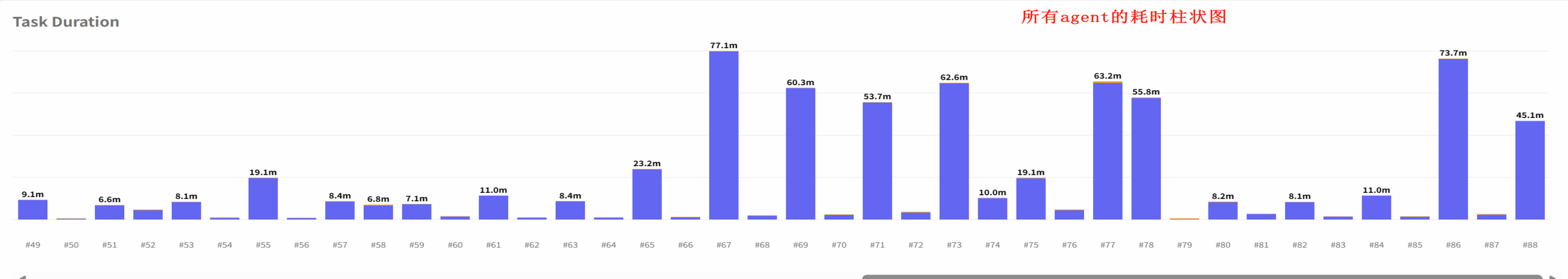
Context: 监控增长趋势帮助解决上下文管理是长程任务遵从性难题

监控上下文膨胀并提前预警：可视化追踪轮次增长，智能划分安全阈值区间



Context面板会以动态图表直观呈现上下文随对话轮次的累积趋势，还会自动将状态划分为安全、警戒、危险三个等级，帮开发者实时掌控上下文负荷，从源头避免模型因窗口超限“失忆”或性能衰减。

Perf：帮助topdown方式分析长程任务中瓶颈



SUBAGENT

▼ kernel

Task 26: Branch-0 Kernel

查看最耗时的任务

8.6%

1h17m

92.2M

#1881

45.2

#1979

2m3s (3%)

179.3K

Cache 99.46%

TPS 99.8

1 tools • cannbot/glm-5.2

2m3s

LLM思考

编写

1m14s (60%)

49.1s (40%)

无工具纯推理

write/edit 文件修改

区分工具时间和模型时间，点击可以调整查看具体turn

▼ 活动

#2014

1m50s (2%)

247.9K

Cache 99.04%

TPS 98.1

1 tools • cannbot/glm-5.2

► 活动

#1975

1m25s (2%)

158.2K

Cache 98.73%

TPS 78.7

1 tools • cannbot/glm-5.2

► 活动

#1938

1m24s (2%)

108.5K

Cache 96.32%

TPS 75.1

1 tools • cannbot/glm-5.2

► 活动

#1954

1m23s (2%)

136.0K

Cache 98.56%

TPS 78.4

1 tools • cannbot/glm-5.2

► 活动

#1976

1m9s (1%)

165.2K

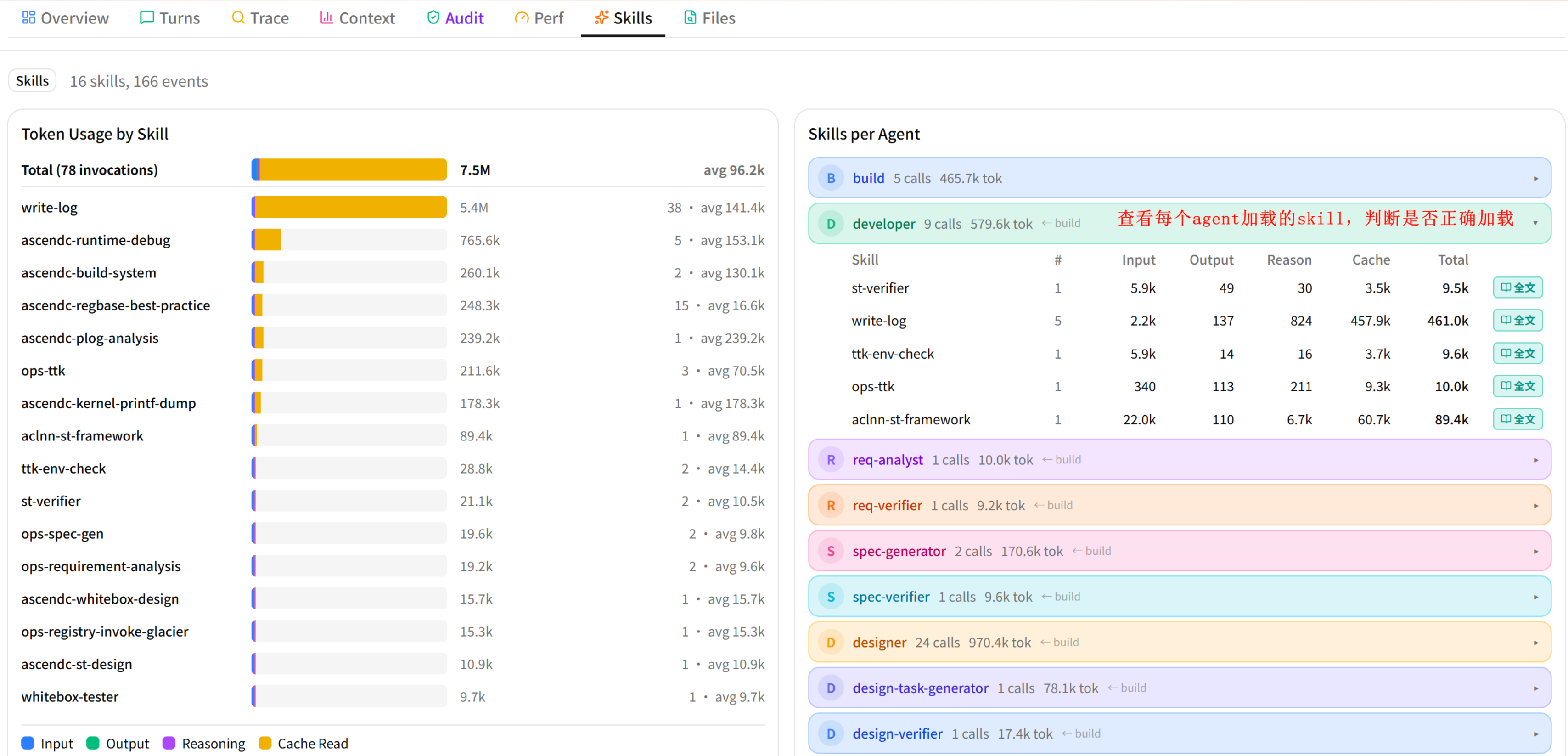
Cache 94.95%

TPS 81.2

1 tools • cannbot/glm-5.2

► 活动

Skills：帮助判断不同子agent中skill是否正确加载



Skills：帮助判断不同子agent中skill是否正确加载

Skill Summary

Skill Name	Version	Invocations	Success	Fail	Avg Duration	Input Tok	Output Tok	Reason Tok	Cache Read Tok	Total Tok	全文	对账
▼ aclnn-st-framework	N/A	1	1	0	N/A	22.0k	110	6.7k	60.7k	89.4k	📄 全文	skill ↗
Turn 1766 • Agent: developer (sub) invoke ok 89.4k tok (22.0k in / 110 out / 6.7k reason / 60.7k cache)												

▶ ascendc-build-system	N/A	2	2	0	N/A	6.3k	66	8.2k	245.6k	260.1k	📄 收起	skill ↗
📄 ascendc-build-system • SKILL.md (来源：Skill 工具注入，3602 字符，框架注入全文) 按时间顺序取本会话内该 skill 的注入内容或 SKILL.md 读取结果；若多次加载，取最长版本。仅含已采集内容，非运行时实时文件。												

点击全文可以导出skill全文

```
<skill_content name="ascendc-build-system">
# Skill: ascendc-build-system

# Ascend C 构建系统

## 心智模型

**真正的编译单元是副本。** 你打开的 `op_kernel/*_apt.cpp` 从不被直接编译：codegen 先生成 `<OpType>.py`，由它驱动 `asc_opc` 去编译源码副本。报错行号属于这条链路上的某一层，定位前先建立
source map——这个行号是哪一层的。

**`DDTYPE_X` 是构建系统注入的宏。** 值由 `_param.json` 决定，编译时以 `-DDTYPE_X=float|half|bfloat16_t` 注入，源码里查不到它展开成什么。同一个 `.cpp` 按 dtype 数量编译多次，每次宏值不同。

**编译是二重枚举，外层 dtype、内层 TPL。** 外层来自 `op_host/*_def.cpp` 的 `.DataType({...})`，每个 dtype 产出一个 `.o`；内层来自 `op_kernel/*_struct.h` 的 `ASCENDC_TPL_SEL`，每行产
出 `.o` 内的一个 sub-kernel。

```text
.o 数量 = 外层 = DataType 列表长度
sub-kernel 数 = 外层 × 内层 = DataType 列表长度 × ASCENDC_TPL_SEL 行数
```

## 完成判据

每次构建之后：

```bash
bash build.sh 2>&1 | tee build.log
```

# audit：对 workflow 进行审计，识别长程任务中的问题指导 skill 优化

← Back

CANNBot-Insight v1.58

Session: "/ops-registry-invoke-glacier ...

刷新

🔄

📄

Tool: cannbot-insight

Model: cannbot/glm-5.2

Source: /home/guan/code/03\_cannbot\_insight/skills-d...

Start: 2026/7/30 00:09

End: 2026/7/30 15:08

Overview

Turns

Trace

Context

Audit

Perf

Skills

Files

Workflow

Skill

分析数据存于 localStorage（按 session 隔离）。

v1

v2

v3

v4

重新生成审计报告

从文件导入

导出到文件

清除

主 agent 因在 #79 完成 Task 19 后即终止、未启动 Task 20-50 的 kernel 实现阶段，整体交付未完成；60 个子 agent 中 5 个 fail/weak（kernel 实现反复试错且最终编译失败/运行时 ACL 507035 错误未解决，verifier 遗漏勾选 STATE.md），其余 55 个 pass。

通过 194

瑕疵 55

未达标 18

不适用 0

agents 89

显示全部通过的子 agent

已隐藏 44

主 agentmain

完成未达标▶ 诊断/建议

依据：#79 read STATE.md 显示 Task 19 已完成、Task 20+（31 个设计驱动任务）已插入但均为 `[ ]` 未执行；envelope turnCount=111 但 actions 仅记录到 #79，后续无任何动作；outputSummary 为空、artifacts 为空，核心产出（kernel 实现）未交付。  
主 agent 作为编排者，仅完成了前置准备与设计阶段（Task 1-19），核心的 kernel 实现阶段（Task 20-50）尚未启动即终止，整体交付物未完成。

耗时 41123.7s

token 7947.4K

turn 111

工具 111

错误 1

重试 1

推理 27.3K

turns: #0 #1 #2 #3 #4 #5 #6 #7 #8 #9 #10 #11 #12 #13 +97

▶ 展开 I/O

▶ 展开动作 (80)

l, main 派发 40 个子 agent

verifier ses\_0515b568cffeEqAwGVc52jWx79

完成通过

依据：#116 bash `ls -ld .../develop/` 返回 `drwxrwxr-x ... develop/` 且输出 `EXISTS\_ACCESSIBLE`，目录存在且可访问，验收标准达成。

耗时 23.6s

token 161.9K

turn 6

工具 5

错误 0

重试 0

推理 0.4K

turns: #115 #116 #117 #118 #130 #133

▶ 展开 I/O

▶ 展开动作 (5)

l, verifier 派发 2 个子 agent

tiling ses\_04ea8f238ffeBa2HkPVcJiKJ1R 工件 1

完成通过

依据：#116 bash 验证命令简洁准确，`ls -ld` + `echo EXISTS\_ACCESSIBLE` 模式清晰判定目录存在性与可访问性；envelope `errorCount=0`、`retryCount=0`，无错误无重试。

耗时 23.6s

token 161.9K

turn 6

工具 5

错误 0

重试 0

推理 0.4K

turns: #115 #116 #117 #118 #130 #133

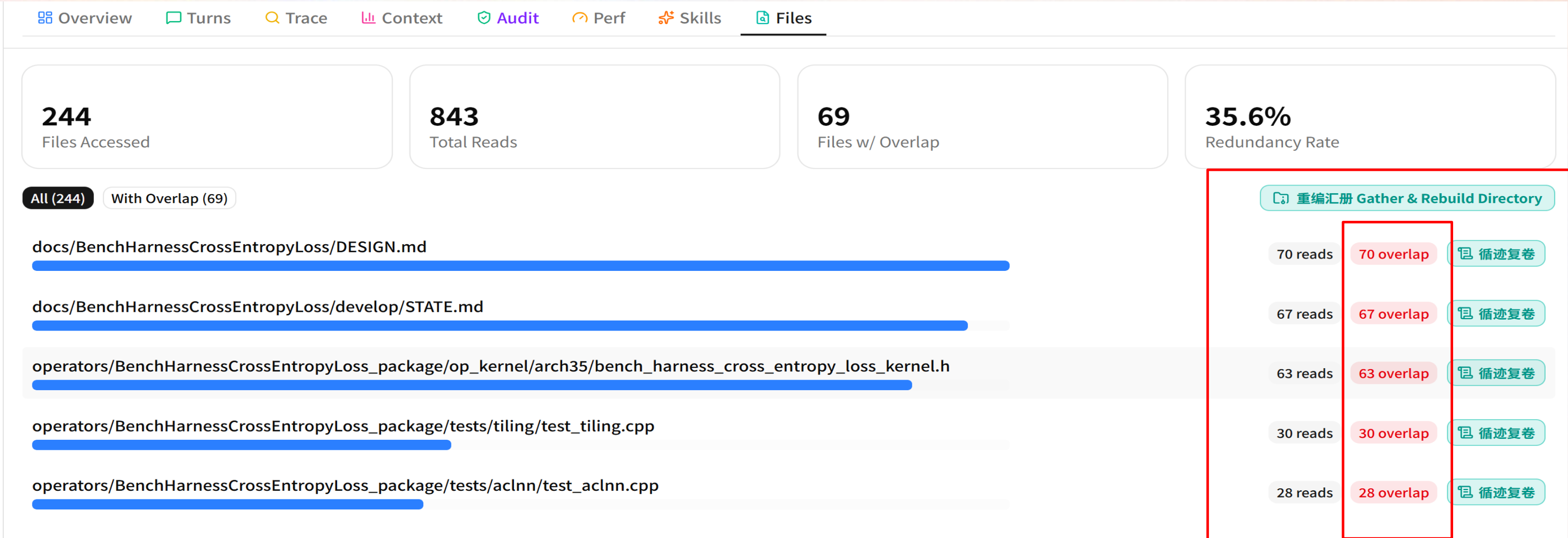
▶ 展开 I/O

▶ 展开动作 (5)

l, verifier 派发 2 个子 agent



# Files 文件访问分析，循迹复卷恢复文件内容



文件重复访问分析

# Files 文件访问分析，循迹复卷恢复中间状态文件内容

docs/BenchHarnessCrossEntropyLoss/develop/STATE.md

67 reads 67 overlap 循迹复卷

operators/BenchHarnessCrossEntropyLoss\_package/op\_kernel/arch35/bench\_harness\_cross\_entropy\_loss\_kernel.h

63 reads 63 overlap 已复卷 · 收起

循迹复卷 · 948 行 (74 次读写, 0 行未采集) ⓘ

按时间顺序拼合本会话中该文件的读写片段，后写入者覆盖先前内容。局部改写 (str\_replace) 未纳入，未被读取的行标注为「未采集」，故重建结果未必等于任一时刻的真实文件。

循迹复卷：恢复中间状态文件

Download ×

```
1 #ifndef BENCH_HARNESS_CROSS_ENTROPY_LOSS_KERNEL_H
2 #define BENCH_HARNESS_CROSS_ENTROPY_LOSS_KERNEL_H
3
4 // =====
5 // BenchHarnessCrossEntropyLoss - Branch-0 Kernel (arch35 / DAV_3510)
6 // DESIGN-BRANCH-0.md §2-§6: C-single-pass / reduction=none.
7 //
8 // Architecture:
9 // CopyIn(DB=2) → ComputeC → CopyOut
10 //
11 // ComputeC for hard labels:
12 // 1. Pre-cast logits D_T → fp32 (into castFp32Buf TBuf, ALWAYS - Gather source)
13 // 2. Gather x_target from fp32 logits (offsets in BYTES per AscendC::Gather ABI)
14 // 3. asc_vf_call: LSE (x_target subtracted post-VF)
15 // 4. Post-VF: p = lse - x_target; ignore_index mask; +inf specialization
16 //
17 // ComputeC for soft labels:
18 // 1. Pre-cast logits + target D_T → fp32
19 // 2. asc_vf_call: LSE + weighted sum + p = lse*sum_w - sum_wx + NaN
20 //
21 // VF accumulators use MaskMergeMode::MERGING on tail chunks (ZEROING would
22 // zero the accumulator lanes for masked elements, corrupting large-C sums).
23 // =====
```



# 实操演示



# CANNBot-Insight的未来规划

## 01 Workflow 全链路动态审计

我们将引入动态审计机制，对工作流的节点执行、权限调用和数据流转做全量记录与实时校验，确保业务合规可追溯。该功能能在异常发生时快速自动定位根因，计划于2026年第三季度正式落地。

## 02 Skill 标准化评测体系

我们会构建“静态分析+动态评测”的双重评测体系模型，从skill质量、安全合规、性能效率等维度，全方位评估技能插件。这套体系预计2026年第三季度上线。

## 开源共建：邀请大家携手同行，共建生态

CANNBot-Insight 项目已在 GitCode 社区正式开源，我们期待与全球开发者共建智能协作生态。无论是功能优化建议、新特性贡献，还是行业场景插件开发，都欢迎通过社区仓库参与讨论与提交，让智能机器人赋能更多业务场景。

开源地址：<https://gitcode.com/cann/cannbot-skills/tree/master/plugins-community/cannbot-insight>



THANKS



参与互动答疑贴赢取礼品





轻量开发专场

# CANN开发者Meetup

茶歇时间，精彩即将继续，敬请期待



主办单位：CANN开源社区    协办单位： AtomGit    开放原子紫金专区

# CANN